ShellLab

计科210X 甘晴void 202108010XXX

写在前面

作为一份实验报告，我希望阅读者能够比较好地看到这份报告有价值的部分。私以为更为有价值的部分体现在：

- 报告中打★的部分，
- 最后的代码中，我在代码中加入了大量的中文注释、函数原型、参数解读、以及个人对代码的推断与理解。
- 实验验证的自动验证部分，这里通过修改makefile文件达成自动化的效果

此外希望阅读者能够看到，若能如此，也算如愿了。

目录

ShellLab
写在前面
目录
0 准备工作
阅读first-shell-lab.txt
解压材料
makefile文件
trace*.txt文件
总结与开始
准备知识
实验目的
实验环境
1 实验内容
1.1 设定与main函数
1.2 void eval(char *cmdline);
函数作用
实现思路
★特别注意
代码实现

1.3 int builtin_cmd(char **argv);
函数作用
实现思路
代码实现
1.4 void do_bgfg(char **argv);
函数作用
实现思路
★特别注意
代码实现
1.5 void waitfg(pid_t pid);
函数作用
实现思路
★特别注意
代码实现
1.6 void sigchld_handler(int sig);
函数作用
实现思路
代码实现
1.7 void sigint_handler(int sig);
函数作用
实现思路
代码实现
1.8 void sigtstp_handler(int sig);
函数作用
实现思路
代码实现
2 实验验证
1 单个验证
（1）trace01-正确地终止EOF
（2）trace02-进程内置quit命令
（3）trace03-运行一个前台job
（4）trace04-运行后台job
（5）trace05-处理jobs内置命令
（6）trace06-将SIGINT转发到前台作业
（7）trace07-仅将SIGINT转发给前台作业
（8）trace08-仅将SIGTSTP转发到前台作业
（9）trace09-进程bg内置命令
（10）trace10-进程fg内置命令
（11）trace11-将SIGINT转发给前台进程组中的每个进程
（12）trace12-将SIGTSTP转发到前台进程组中的每个进程
（13）trace13-重新启动进程组中的每个已停止的进程
（14）trace14-简单的错误处理
（15）trace15-综合测试

（16）trace16-来自其他进程的信号
★2 自动验证
（1）修改makefile文件
（2）测试所有测试集并使用重定向符将其导出
（3）使用meld进行比对
★3 实验总结
★1 函数视角
★2 信号视角
★并发问题
4 完整代码
5 学习参考

0 准备工作

我们拿到手上的文件包解压之后有三个文件

- readme first-shell-lab.txt
- shlab-handout
- shlab-overview.pdf

阅读first-shell-lab.txt

第一个是老师为我们准备的贴心提示，特别友好地使用中文列出了我们需要干的事情，这也是这个实验中我们唯一能见到中文的地方。

```
//readme first-shell-lab.txt
这个实验是大家在本课程第一次体验系统级编程，涉及过程，过程控制和信号的相关知识。
1.你需要干什么？
      你需要构建一个简单的类Unix/Linux Shell。基于已经提供的“微Shell”框架
tsh.c，完成部分函数和信号处理函数的编写工作。使用sdriver.pl可以评估你所完成的
shell的相关功能。


2. 准备工作
  使用命令tar xvf shelab-handout.tar 解压缩文件；
  使用命令 make   去编译和链接一些测试例程；
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  你要实现的重要函数列出如下：
```

```
  eval            主例程，用以分析和解释命令行（好消息：该函数原型在教材一第8章8.4
节中可以找到！）；
  builtin_cmd   执行bg和fg内置命令；
  waitfg          等待前台作业执行；
  sigchld_handler   响应处理SIGCHILD信号
  sigint_handler    响应处理SIGINT（ctrl-c）信号
  sigtstp_handler   相应处理SIGSTP（ctrl-z）信号

3.注意
  每次修改了tsh.c文件，都需要make它以重新编译。在你的Linux终端中直接运行tsh
（./tsh）就可以进入你所编写完成的tiny shell tsh>了。

4. 如何证明你完成了这个实验
   在你的Linux终端运行./tshref  这个已经实现的shell，将其输出结果与你所实现
的./tsh 输出结果比较，是否一致。
相关比较命令行，参见shelab-overview文件。

5.请在实验报告体现你解决本实验目标的详细过程，仅仅贴图（图中只有代码）可能会导致
“无个人工作，仅仅是复制粘贴”的极低分判定。

Love & Peace!
```

解压材料

第二个是我们实验的环境以及材料，但需要解压。

使用tar指令解压包

```
tar xvf shlab-handout.tar
```

解压完成之后我们可以看到实验的材料

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ ls
Makefile    README       trace04.txt  trace09.txt  trace14.txt
tshref.out
myint.c     sdriver.pl   trace05.txt  trace10.txt  trace15.txt
myspin.c    trace01.txt  trace06.txt  trace11.txt  trace16.txt
mysplit.c   trace02.txt  trace07.txt  trace12.txt  tsh.c
mystop.c    trace03.txt  trace08.txt  trace13.txt  tshref
```

打开这里的README，它会解释这些文件的作用。

```
################
CS:APP Shell Lab
################

Files:
Makefile    # Compiles your shell program and runs the tests
README      # This file
tsh.c       # The shell program that you will write and hand in（这是我们要完善的程序）
tshref      # The reference shell binary.

# The remaining files are used to test your shell
sdriver.pl  # The trace-driven shell driver
trace*.txt  # The 15 trace files that control the shell driver
tshref.out  # Example output of the reference shell on all 15 traces

# Little C programs that are called by the trace files
myspin.c    # Takes argument <n> and spins for <n> seconds
mysplit.c   # Forks a child that spins for <n> seconds
mystop.c        # Spins for <n> seconds and sends SIGTSTP to itself
myint.c         # Spins for <n> seconds and sends SIGINT to itself
```

makefile文件

接下来我们查看makefile文件

```
# Makefile for the CS:APP Shell Lab

TEAM = NOBODY
VERSION = 1
HANDINDIR = /afs/cs/academic/class/15213-f02/L5/handin
DRIVER = ./sdriver.pl
TSH = ./tsh
TSHREF = ./tshref
TSHARGS = "-p"
CC = gcc
CFLAGS = -Wall -O2
FILES = $(TSH) ./myspin ./mysplit ./mystop ./myint
```

```
all: $(FILES)

##################
# Handin your work
##################
handin:
    cp tsh.c $(HANDINDIR)/$(TEAM)-$(VERSION)-tsh.c


##################
# Regression tests
##################

# Run tests using the student's shell program
test01:
    $(DRIVER) -t trace01.txt -s $(TSH) -a $(TSHARGS)
test02:
    $(DRIVER) -t trace02.txt -s $(TSH) -a $(TSHARGS)
test03:
    $(DRIVER) -t trace03.txt -s $(TSH) -a $(TSHARGS)
test04:
    $(DRIVER) -t trace04.txt -s $(TSH) -a $(TSHARGS)
test05:
    $(DRIVER) -t trace05.txt -s $(TSH) -a $(TSHARGS)
test06:
    $(DRIVER) -t trace06.txt -s $(TSH) -a $(TSHARGS)
test07:
    $(DRIVER) -t trace07.txt -s $(TSH) -a $(TSHARGS)
test08:
    $(DRIVER) -t trace08.txt -s $(TSH) -a $(TSHARGS)
test09:
    $(DRIVER) -t trace09.txt -s $(TSH) -a $(TSHARGS)
test10:
    $(DRIVER) -t trace10.txt -s $(TSH) -a $(TSHARGS)
test11:
    $(DRIVER) -t trace11.txt -s $(TSH) -a $(TSHARGS)
test12:
    $(DRIVER) -t trace12.txt -s $(TSH) -a $(TSHARGS)
test13:
    $(DRIVER) -t trace13.txt -s $(TSH) -a $(TSHARGS)
test14:
```

```
	$(DRIVER) -t trace14.txt -s $(TSH) -a $(TSHARGS)
test15:
	$(DRIVER) -t trace15.txt -s $(TSH) -a $(TSHARGS)
test16:
	$(DRIVER) -t trace16.txt -s $(TSH) -a $(TSHARGS)

# Run the tests using the reference shell program
rtest01:
	$(DRIVER) -t trace01.txt -s $(TSHREF) -a $(TSHARGS)
rtest02:
	$(DRIVER) -t trace02.txt -s $(TSHREF) -a $(TSHARGS)
rtest03:
	$(DRIVER) -t trace03.txt -s $(TSHREF) -a $(TSHARGS)
rtest04:
	$(DRIVER) -t trace04.txt -s $(TSHREF) -a $(TSHARGS)
rtest05:
	$(DRIVER) -t trace05.txt -s $(TSHREF) -a $(TSHARGS)
rtest06:
	$(DRIVER) -t trace06.txt -s $(TSHREF) -a $(TSHARGS)
rtest07:
	$(DRIVER) -t trace07.txt -s $(TSHREF) -a $(TSHARGS)
rtest08:
	$(DRIVER) -t trace08.txt -s $(TSHREF) -a $(TSHARGS)
rtest09:
	$(DRIVER) -t trace09.txt -s $(TSHREF) -a $(TSHARGS)
rtest10:
	$(DRIVER) -t trace10.txt -s $(TSHREF) -a $(TSHARGS)
rtest11:
	$(DRIVER) -t trace11.txt -s $(TSHREF) -a $(TSHARGS)
rtest12:
	$(DRIVER) -t trace12.txt -s $(TSHREF) -a $(TSHARGS)
rtest13:
	$(DRIVER) -t trace13.txt -s $(TSHREF) -a $(TSHARGS)
rtest14:
	$(DRIVER) -t trace14.txt -s $(TSHREF) -a $(TSHARGS)
rtest15:
	$(DRIVER) -t trace15.txt -s $(TSHREF) -a $(TSHARGS)
rtest16:
	$(DRIVER) -t trace16.txt -s $(TSHREF) -a $(TSHARGS)

# clean up
clean:
```

```
    rm -f $(FILES) *.o *~
```

可以看到makefile内帮我们写好了相应的命令。由于是线下验收，故不需要考虑all和handin指令。

这里比较重要的是`test*`指令，`rtest*`指令和`clean`指令。（*表示01到16的其中一个数）

使用`test*`指令可以用自己写的程序运行tinyshell，使用`rtest*`指令可以使用参考的可执行文件看看生成了什么，用这个可以跟我们自己生成的进行比对，从而检验我们是不是写对了。每次改动tsh.c之后都要进行make clean重新生成。

trace*.txt文件

随便打开一个trace文件。

```
#
# trace04.txt - Run a background job.
#
/bin/echo -e tsh> ./myspin 1 \046
./myspin 1 &
```

看到这实际上是一个指令的集合，用于测试我们的tinyshell。

总结与开始

至此，我们要干的事情以已经十分明了了。

在tsh.c中有一些函数（7个）待实现，其他函数已经实现了。

我们要实现这7个函数，然后有16个测试指令集分别对我们的tinyshell进行功能验证。

在编写完函数后，在终端中使用如下方式验证

```
#这里的*要被换成相应的数字
make test*      #这是运行我们自己的tinyshell
make rtest*     #这是使用参考的tinyshell
#如果两者输出一致，基本可以认为这个测试点通过了
```

在我们使用`make test*`后，make会调用相应的路径启动相关的测试指令集并完成该测试点的测试。

全部测试点通过，我们的tinyshell可以被认为基本搭建正确。

准备知识

进程的概念、状态以及控制进程的几个函数（fork,waitpid,execve）。
信号的概念，会编写正确安全的信号处理程序。
shell的概念，理解shell程序是如何利用进程管理和信号去执行一个命令行语句。

实验目的

shell lab主要目的是为了熟悉进程控制和信号。实现一个简单的shell，能够处理前后台运行程序、能够处理ctrl+z、ctrl+c等信号，实现七个函数：

```
void eval(char *cmdline);         //分析命令，并派生子进程执行 主要功能是解析cmdline并运行
int builtin_cmd(char **argv);     //解析和执行bulidin命令，包括 quit, fg, bg, and jobs
void do_bgfg(char **argv);        //执行bg和fg命令
void waitfg(pid_t pid);           //实现阻塞等待前台程序运行结束
void sigchld_handler(int sig);    //SIGCHID信号处理函数
void sigint_handler(int sig);     //信号处理函数，响应 SIGINT (ctrl-c) 信号
void sigtstp_handler(int sig);    //信号处理函数，响应 SIGTSTP (ctrl-z) 信号
```

实验环境

ubuntu12.04 (32位)环境

1 实验内容

1.1 设定与main函数

定义一些宏

```
/* Misc manifest constants */
#define MAXLINE    1024   /* max line size */
#define MAXARGS     128   /* max args on a command line */
#define MAXJOBS      16   /* max jobs at any point in time */
#define MAXJID    1<<16   /* max job ID */
```

定义4种工作状态

```
/* Job states */
#define UNDEF 0 /* undefined */
#define FG 1    /* running in foreground */
#define BG 2    /* running in background */
#define ST 3    /* stopped */
```

定义job_t的任务的类，创建jobs[]数组，这是全局变量

```
/*
 * Jobs states: FG (foreground), BG (background), ST (stopped)
 * Job state transitions and enabling actions:
 *     FG -> ST  : ctrl-z
 *     ST -> FG  : fg command
 *     ST -> BG  : bg command
 *     BG -> FG  : fg command
 * At most 1 job can be in the FG state.
 */
/* Global variables */
struct job_t {              /* The job struct */
    pid_t pid;              /* job PID */
    int jid;                /* job ID [1, 2, ...] */
    int state;              /* UNDEF, BG, FG, or ST */
    char cmdline[MAXLINE];  /* command line */
};
struct job_t jobs[MAXJOBS]; /* The job list */
```

定义其他的全局变量

```
/* Global variables */
extern char **environ;      /* defined in libc */
char prompt[] = "tsh> ";    /* command line prompt (DO NOT CHANGE)
*/
int verbose = 0;            /* if true, print additional output */
int nextjid = 1;            /* next job ID to allocate */
char sbuf[MAXLINE];         /* for composing sprintf messages */
/* End global variables */
```

接下来开始定义函数

```
/* Function prototypes */
```

定义我们需要补全的函数

```
/* Here are the functions that you will implement */
void eval(char *cmdline);         //分析命令，并派生子进程执行 主要功能是解
析cmdline并运行
int builtin_cmd(char **argv);     //解析和执行bulidin命令，包括 quit,
fg, bg, and jobs
void do_bgfg(char **argv);        //执行bg和fg命令
void waitfg(pid_t pid);           //实现阻塞等待前台程序运行结束

void sigchld_handler(int sig);    //SIGCHID信号处理函数
void sigint_handler(int sig);     //信号处理函数，响应 SIGINT (ctrl-c)
信号
void sigtstp_handler(int sig);    //信号处理函数，响应 SIGTSTP (ctrl-z)
信号
```

定义已经实现并提供给我们的函数

```
/* Here are helper routines that we've provided for you */
int parseline(const char *cmdline, char **argv); //获取参数列表，返回是
否为后台运行命令
void sigquit_handler(int sig);//处理SIGQUIT信号

void clearjob(struct job_t *job);//清除job结构体
void initjobs(struct job_t *jobs);//初始化任务jobs[]
int maxjid(struct job_t *jobs); //返回jobs链表中最大的jid号。
```

```
int addjob(struct job_t *jobs, pid_t pid, int state, char
*cmdline);//向jobs[]添加一个任务
int deletejob(struct job_t *jobs, pid_t pid);//在jobs[]中删除pid的job
pid_t fgpid(struct job_t *jobs);//返回当前前台运行job的pid号
struct job_t *getjobpid(struct job_t *jobs, pid_t pid);//根据pid找到
对应的job
struct job_t *getjobjid(struct job_t *jobs, int jid); //根据jid找到对
应的job
int pid2jid(pid_t pid);  //根据pid找到jid
void listjobs(struct job_t *jobs);//打印jobs

void usage(void);
void unix_error(char *msg);
void app_error(char *msg);
typedef void handler_t(int);
handler_t *Signal(int signum, handler_t *handler);
```

接下来是main函数

```
/*
 * main - The shell's main routine
 */
int main(int argc, char **argv)
{
    char c;
    char cmdline[MAXLINE];
    int emit_prompt = 1; /* emit prompt (default) */

    /* 将stderr重定向到stdout（这样驱动程序将获得连接到stdout的管道上的所有输
出） */
    dup2(1, 2);

    /* 解析命令行 */
    while ((c = getopt(argc, argv, "hvp")) != EOF)
    {
        switch (c)
        {
        case 'h': /* 打印提示信息 */
            usage();
            break;
        case 'v': /* 发出附加诊断信息 */
```

```
            verbose = 1;
            break;
        case 'p':            /* 不打印提示 */
            emit_prompt = 0; /* 便于自动测试 */
            break;
        default:
            usage();
        }
    }

    /* Install the signal handlers */

    /* These are the ones you will need to implement */
    Signal(SIGINT, sigint_handler);   /* ctrl-c */
    Signal(SIGTSTP, sigtstp_handler); /* ctrl-z */
    Signal(SIGCHLD, sigchld_handler); /* Terminated or stopped
child */

    /* This one provides a clean way to kill the shell */
    Signal(SIGQUIT, sigquit_handler);

    /* 初始化 job list */
    initjobs(jobs);

    /* Execute the shell's read/eval loop */
    while (1)
    {

        /* Read command line */
        if (emit_prompt)
        {
            printf("%s", prompt);
            fflush(stdout); // 清空缓存区
        }
        if ((fgets(cmdline, MAXLINE, stdin) == NULL) &&
ferror(stdin))
            app_error("fgets error");
        if (feof(stdin))
        { /* End of file (ctrl-d) */
            fflush(stdout);
            exit(0);
        }
```

```
        /* Evaluate the command line */
        eval(cmdline);
        fflush(stdout);
        fflush(stdout);
    }

    exit(0); /* control never reaches here */
}
```

mian函数，作用是在文件中逐行获取命令，并且判断是不是文件结束（EOF），将命令cmdline送入eval函数进行解析。

可以预见，若没有结束命令，这个main函数将持续进行，不断对用户输入做出反应。

1.2 void eval(char *cmdline);

函数作用

- 解析和解释命令行。
- eval首先解析命令行，如果用户请求了一个内置命令（quit、jobs、bg或fg），则立即执行。
- 否则，派生一个子进程并在该子进程的上下文中运行作业。
- 如果作业正在前台运行，请等待它终止，然后返回。

注意：每个子进程都必须有一个唯一的进程组ID，这样当我们在键盘上键入ctrl-c（ctrl-z）时，我们的后台子进程就不会从内核接收SIGINT（SIGTSTP）。

实现思路

- 初始化/声明变量
- 调用parseline函数解析命令行，解析结果放入argv，该函数返回值表示新进程的前台/后台状态。（若空命令行则返回）
- 调用函数builtin_cmd(argv)执行内置命令（quit,bg,fg,jobs），若命令不是内置命令，该函数返回0
- 若不是内置命令，则调用fork新建子进程
- 新建子进程后，父进程通过addjob将子进程的信息加入jobs，子进程通过execve函数执行新的任务
- 若子进程在前台作业，调用waitfg等待子进程终止

★特别注意

- 屏蔽信号量：主要是防止fork之后调度执行子进程并在addjob之前结束子进程，此时SIGCHLD信号使父进程将子进程回收，这会导致 addjob 和 deletejob 函数执行错位，结果是删除一个不存在的进程号，添加一个不存在且永不会被删除的进程号。
- 调用parseline、sigemptyset、sigaddset、sigprocmask等函数，由于这些函数提供特殊返回值校验，即若函数没有完成既定目的则返回-1，所以一定在函数返回后进行判定。推荐直接将调用过程写在if判定语句内。

代码实现

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; // execve()函数的参数
    int state = UNDEF;   // 工作状态，FG或BG
    sigset_t set;
    pid_t pid; // 进程id

    // 处理输入的数据
    if (parseline(cmdline, argv) == 1) // 解析命令行，返回给argv数组
        state = BG;
    else
        state = FG;
    if (argv[0] == NULL) // 若命令行为空，parseline也会返回1，但对argv[0]判定后，eval在这里直接返回
        return;

    // 如果不是内置命令
    if (!builtin_cmd(argv)) // 若是内置命令，builtin_cmd(argv)会执行，若该函数返回0，则表示非内置命令
    {
        // 初始化信号集set并把SIGINT SIGTSTP SIGCHLD三个信号放入信号集中，方便管理
        if (sigemptyset(&set) < 0)
            unix_error("sigemptyset error");
        if (sigaddset(&set, SIGINT) < 0 || sigaddset(&set, SIGTSTP) < 0 || sigaddset(&set, SIGCHLD) < 0)
            unix_error("sigaddset error");

        // 阻塞SIGCHLD信号，具体原因在书P541有解释
```

```
        // 主要是防止fork之后调度执行子进程并在addjob之前结束子进程，
        // 此时SIGCHLD信号使父进程将子进程回收，这会导致 addjob 和
deletejob 函数执行错位
        // 结果是删除一个不存在的进程号，添加一个不存在且永不会被删除的进程号
        if (sigprocmask(SIG_BLOCK, &set, NULL) < 0)
            unix_error("sigprocmask error");

        if ((pid = fork()) < 0) // fork创建子进程失败
            unix_error("fork error");
        else if (pid == 0) // fork创建子进程
        {
            // 子进程控制流从这里开始

            if (sigprocmask(SIG_UNBLOCK, &set, NULL) < 0) // 解除阻塞
                unix_error("sigprocmask error");

            // 函数原型 int setpgid(pid_t pid,pid_t pgid);
            // 将pid进程的进程组ID设置成pgid
            // 如果参数pid为0，则会用来设置该进程的组识别码，
            // 如果参数pgid为0，则由pid指定的进程ID将用作进程组ID
            // 一个进程只能为它自己或它的子进程设置进程组ID,不能为其父进程设置
ID。
            if (setpgid(0, 0) < 0) // 设置子进程id（实际上并没有分进程组，
因为一人一组）
                unix_error("setpgid error");

            if (execve(argv[0], argv, environ) < 0)
            {
                printf("%s: command not found\n", argv[0]);
                exit(0);
            }
        }

        // 父进程控制流从这里开始
        addjob(jobs, pid, state, cmdline); // 将当前进程添加进jobs中，
参数为当前进程pid,state,cmdline

        // 恢复受阻塞的信号 SIGINT SIGTSTP SIGCHLD
        if (sigprocmask(SIG_UNBLOCK, &set, NULL) < 0)
            unix_error("sigprocmask error");

        // 判断子进程类型并做处理
```

```
        if (state == FG)
            waitfg(pid); // 等待子进程的前台作业完成
        else
            printf("[%d] (%d) %s", pid2jid(pid), pid, cmdline); //
将进程id映射到job id
    }
    return;
}
```

1.3 int builtin_cmd(char **argv);

函数作用

解析和执行bulidin命令，包括 quit, fg, bg, and jobs

实现思路

- 若命令为quit，调用exit实现退出
- 若命令为bg或fg，调用do_fgbg函数执行相关操作
- 若命令为jobs，调用listjobs函数打印后台作业
- 若都不满足，则不是内置命令，以0返回eval，eval会做相应的处理

代码实现

```
int builtin_cmd(char **argv)
{
    if (!strcmp(argv[0], "quit")) // 如果命令是quit，退出
        exit(0);
    else if (!strcmp(argv[0], "bg") || !strcmp(argv[0], "fg")) //
如果是bg或者fg命令，执行do_fgbg函数
        do_bgfg(argv);
    else if (!strcmp(argv[0], "jobs")) // 如果命令是jobs，列出正在运行和
停止的后台作业
        listjobs(jobs);
    else
        return 0; /* 不是内置命令，以0返回eval */
    return 1;
}
```

1.4 void do_bgfg(char **argv);

函数作用

完成内置指令bg或fg的切换进程前后台状态的操作。

实现思路

- 首先监测bg/fg后面有没有参数，若命令行为空，则输出指令错误并返回
- 通过读取命令行的第一个字符是否为%来判断ID类型是jobID还是PID
- 两种情况都先调用strtol函数获取ID编号，然后分别调用getjobjid和geijobpid获取job本体
- 判断指令是bg、fg还是其他
- 若为bg，则设置状态BG，调用kill函数向该进程组发送继续运行信息，并向终端输出信息（进程在后台继续运行）
- 若为fg，则设置状态FG，调用kill函数向该进程组发送继续运行信息，并调用waitfg等待该进程结束（等待前台进程结束）
- 若为其他，则输出错误信息并终止该进程（这个命令不对，不能继续进行）

★特别注意

- 几个函数（strtol和kill等）的原型很重要，建议了解。
- kill函数还是要进行返回值判定该kill有没有正常执行，建议使用if语句条件中调用kill函数

代码实现

```
void do_bgfg(char **argv)
{
    int num;
    struct job_t *job;
    // 没有参数的fg/bg不符合规定
    if (!argv[1])
    { // 命令行为空
        printf("%s command requires PID or %%jobid argument\n",
argv[0]);
        return;
    }
```

```
    // strtol函数原型：long int strtol(const char *nptr, char
**endptr, int base);
    // strtol函数会将参数nptr字符串根据参数base来转换成长整型数，参数base范围
从2至36。

    // 检测fg/bg参数，其中%开头的数字是JobID，纯数字的是PID
    // 找到jobID或PID后通过这个找出job
    if (argv[1][0] == '%')
    {                                                   // 解析jid
        if ((num = strtol(&argv[1][1], NULL, 10)) <= 0) // 获取jid
        {
            printf("%s: argument must be a PID or %%jobid\n",
argv[0]); // 失败,打印错误消息
            return;
        }
        if ((job = getjobjid(jobs, num)) == NULL) // 根据jid获取job
        {
            printf("%%%d: No such job\n", num); // 没找到对应的job
            return;
        }
    }
    else
    {                                               // 解析PID
        if ((num = strtol(argv[1], NULL, 10)) <= 0) // 获取PID
        {
            printf("%s: argument must be a PID or %%jobid\n",
argv[0]); // 失败,打印错误消息
            return;
        }
        if ((job = getjobpid(jobs, num)) == NULL) // 根据PID获取job
        {
            printf("(%d): No such process\n", num); // 没找到对应的进
程
            return;
        }
    }

    // kill函数原型： int kill(pid_t pid,int signo)
    // pid > 0：将信号发送给进程 ID 为 pid 的进程。
    // pid ==0：将信号发送给与发送进程属于同一进程组的所有进程。
    // pid < 0：将信号发送给进程组 ID 等于 pid 的绝对值的所有进程。
```

```
    // pid ==-1：将信号发送给系统中所有进程。
    if (!strcmp(argv[0], "bg")) // 该进程需要在后台运行
    {
        // bg会启动子进程，并将其放置于后台执行
        job->state = BG;                    // 设置状态BG
        if (kill(-job->pid, SIGCONT) < 0) // 采用负数发送信号到进程组
            unix_error("kill error");
        printf("[%d] (%d) %s", job->jid, job->pid, job->cmdline);
    }
    else if (!strcmp(argv[0], "fg")) // 该进程需要在前台运行
    {
        job->state = FG;                    // 设置状态FG
        if (kill(-job->pid, SIGCONT) < 0) // 采用负数发送信号到进程组
            unix_error("kill error");
        // 当一个进程被设置为前台执行时，当前tsh应该等待该子进程结束
        waitfg(job->pid);
    }
    else // 指令出现异常
    {
        puts("do_bgfg: Internal error");
        exit(0);
    }
    return;
}
```

1.5 void waitfg(pid_t pid);

函数作用

实现阻塞等待前台程序运行结束。

实现思路

- 通过pid获取该pid对应的job本体
- 新设立一个wait信号集并清空该信号集
- 若前台进程未结束，使用空信号集替换信号掩码，即信号掩码为空，此时任何信号都会唤醒该进程。

★特别注意

- 这里可以使用`job->state == FG`作为判定前台进程是否结束的依据依赖于我们实现的tinyshell中job的储存方式仅仅简单地使用了数组，并且在一开始的时候全部初始化为0，而且删除也仅仅是将其置为0而不是删除掉该片空间。若存储job的方式更改为链表或者使用malloc和free这样进程结束就找不到地方的方法，我们再写这一句话会导致段错误。
- sigsuspend函数的实现以及信号掩码的概念需要对于信号系统有较为深入的理解。特别注意sigsuspend函数所提供的原子性地进入等待所带来的便利性。这个函数解决了一个问题如下：

```
if(sigprocmask(SIG_SETMASK,&oldmask,NULL)<0)
{err_sys("SIG_SETMASK error");}
//------------>此处正好接受并处理SIGINT信号（与内核如何实现信号有关）
pause();
```

> 上述代码试图用*sigprocmask*函数和*pause*函数来达成改变掩码并进入休眠的效果，
>
> 但它万万没想到倘若在这两句本该被认为一起执行的函数之间若进入了别的信号，此时函数*pause*将被永远阻塞，它试图达到的功能将不能被达到。
>
> 此时我们迫切地需要一个由操作系统支持的“恢复信号掩码”和“将进程放到*sleep*状态”能在单个原子操作内完成的操作。

代码实现

```
void waitfg(pid_t pid)
{
    // 通过pid获取该pid对应的job本体
    struct job_t *job = getjobpid(jobs, pid);
    if (!job)
        return;

    // 新设立一个wait信号集
    sigset_t wait;
    if (sigemptyset(&wait) < 0) // 清空wait信号集，该信号集为空
        unix_error("sigemptyset &wait error");

    // sigsuspend函数原型：  int sigsuspend(const sigset_t *mask);
```

```
    // 用于在接收到某个信号之前，临时用mask替换进程的信号掩码，并暂停进程执行，直到收到信号为止

    //    如果当前子进程的状态没有发生改变，则tsh继续休眠
    while (job->state == FG)
        // 一旦有信号改变，就判定是否
        sigsuspend(&wait); // 使用空信号集替换信号掩码，即信号掩码为空，此时任何信号都会唤醒该进程

    return;
}
```

1.6 void sigchld_handler(int sig);

函数作用

sigchld信号处理例程-检测到sigchld信号就调用该程序处理僵尸子进程

僵尸进程的回收由内核完成，这里主要是在jobs中删除僵尸进程的信息并向终端输出操作信息

实现思路

- 使用while调用waitpid函数等待该进程被内核回收，出现子进程被回收的情况就获取该子进程的pid
- 调用getjobpid函数通过待处理子进程的pid获取子进程的job本体，若无法获取，则是发生了问题，输出错误信息并退出
- 根据待处理子进程的状态分三种情况进行讨论
- 若该待处理子进程只是接到了一个暂停信号并没有结束，调用WIFSTOPPED(status)获取使子进程暂停得信号编号并将子进程的状态设为挂起
- 若该待处理子进程通过调用exit或return正常返回，调用WIFEXITED(status)获取退出参数（也就是exit的参数）
- 若该待处理子进程是因为一个未被捕获的信号终止的，则通过WTERMSIG (status)获取使该子进程退出的信号编号

代码实现

```
void sigchld_handler(int sig)
{
    int status, jid;
    pid_t pid;
    struct job_t *job;

    if (verbose)
        puts("sigchld_handler: entering"); // 输出额外信息

    /*
    waitpid函数原型： pid_t waitpid(pid_t pid , int *status , int
options)
    总体：
    如果没有子进程或其它错误原因，则返回-1；
    如果成功回收子进程，则返回回收的那个子进程的ID；
    如果第三个参数为WNOHANG，且子进程都在运行，则返回0
    参数：
    pid：从参数的名字上可以看出来这是一个进程的ID。但是这里pid的值不同时，会有不
同的意义。
        1.pid > 0时，只等待进程ID等于pid的子进程，只要该子进程不结束，就会一直
等待下去；
        2.pid = -1时，等待任何一个子进程的退出，此时作用和wait相同；
        3.pid = 0时，等待同一个进程组中的任何子进程；
        4.pid < -1时，等待一个指定进程组中的任何子进程，这个进程组的ID等于pid
的绝对值。
    options:options提供了一些额外的选项来控制waitpid
        WNOHANG    : 若子进程仍然在运行，则返回0
        （注意只有设置了这个标志，waitpid才有可能返回0）
        WUNTRACED : 如果子进程由于传递信号而停止，则马上返回。
        （只有设置了这个标志，waitpid返回时，其WIFSTOPPED(status)才有可能返
回true）
    &status参数：
        WIFEXITED(status)
        如果正常退出（exit）返回非零值；这时可以用WEXITSTATUS(status) 得到退
出编号（exit的参数）
        WIFSIGNALED(status)
        如果异常退出 （子进程接受到退出信号） 返回非零值；使用WTERMSIG
(status) 得到使子进程退出得信号编号
        WIFSTOPPED(status)
```

```
        如果是暂停进程返回的状态，返回非零值；使用WSTOPSIG（status） 得到使子进程暂停得信号编号
    */

    while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED)) > 0)
// 以非阻塞方式等待所有子进程，若成功回收了子进程，则返回这个子进程的PID，&status中返回其状态
    {

        // 如果当前这个子进程的job已经删除了，则表示有错误发生
        if ((job = getjobpid(jobs, pid)) == NULL)
        {
            printf("Lost track of (%d)\n", pid);
            return;
        }

        jid = job->jid;
        // 接下来判断三种状态
        //  如果这个子进程收到了一个暂停信号（还没退出）
        if (WIFSTOPPED(status))
        {
            printf("Job [%d] (%d) stopped by signal %d\n", jid,
job->pid, WSTOPSIG(status));
            // 使用WSTOPSIG（status） 得到使子进程暂停得信号编号
            job->state = ST; // 状态设为挂起
        }
        // 如果子进程通过调用 exit 或者一个返回 (return) 正常终止
        else if (WIFEXITED(status))
        {
            if (deletejob(jobs, pid))
                if (verbose)
                {
                    printf("sigchld_handler: Job [%d] (%d)
deleted\n", jid, pid);
                    printf("sigchld_handler: Job [%d] (%d)
terminates OK (status %d)\n", jid, pid, WEXITSTATUS(status));
                    // 用WEXITSTATUS(status) 得到退出编号（exit的参数）
                }
        }
        // 如果子进程是因为一个未被捕获的信号终止的，例如SIGKILL
        else
        {
```

```
            if (deletejob(jobs, pid))
            { // 清除进程
                if (verbose)
                    printf("sigchld_handler: Job [%d] (%d)
deleted\n", jid, pid);
            }
            printf("Job [%d] (%d) terminated by signal %d\n", jid,
pid, WTERMSIG(status));
            // 使用WTERMSIG (status) 得到使子进程退出得信号编号
        }
    }

    if (verbose)
        puts("sigchld_handler: exiting");

    return;
}
```

1.7 void sigint_handler(int sig);

函数作用

处理由终端输入ctrl-c引起的异常，将前台作业结束（实现由操作系统内核完成，这里的作用是以内核特权向进程发送SIGINT信号，这个是kill函数做到的）

实现思路

- 获取前台进程的pid
- 若pid合法，调用kill函数发送SIGINT信号给前台进程

代码实现

```
void sigint_handler(int sig)
{
    if (verbose)
        puts("sigint_handler: entering");
    pid_t pid = fgpid(jobs);//获取前台进程的pid
```

```
    if (pid)
    {
        // 发送SIGINT给前台进程组里的所有进程
        // 需要注意的是，前台进程组内的进程除了当前前台进程以外，还包括前台进程的
子进程。
        // 最多只能存在一个前台进程，但前台进程组内可以存在多个进程
        if (kill(-pid, SIGINT) < 0) // 采用负数发送信号到进程组，使该进程
结束
            unix_error("kill (sigint) error");
        if (verbose)
        {
            printf("sigint_handler: Job (%d) killed\n", pid);
        }
    }
    if (verbose)
        puts("sigint_handler: exiting");
    return;
}
```

1.8 void sigtstp_handler(int sig);

函数作用

处理由终端输入ctrl-z引起的异常，将前台作业暂停（实现由操作系统内核完成，这里的作用是以内核特权向进程发送SIGTSTP信号，这个是kill函数做到的）

实现思路

- 获取前台进程的pid
- 调用getjobpid函数，通过前台进程pid获得前台进程job本体
- 若pid合法，调用kill函数发送SIGTSTP信号给前台进程，并输出相关信息。

代码实现

```
void sigtstp_handler(int sig)
{
    if (verbose)
        puts("sigstp_handler: entering");

    pid_t pid = fgpid(jobs);                    // 获取前台进程的pid号
    struct job_t *job = getjobpid(jobs, pid); // 获取前台进程的job本体

    if (pid)

        if (kill(-pid, SIGTSTP) < 0) // 采用负数发送信号到进程组，使该进程
挂起
            unix_error("kill (tstp) error");
    if (verbose)
    {
        printf("sigstp_handler: Job [%d] (%d) stopped\n", job->jid,
pid);
    }

    if (verbose)
        puts("sigstp_handler: exiting");
    return;
}
```

2 实验验证

1 单个验证

之前说过，我们可以单个地进行验证，逐个验证各个模块的正确性。

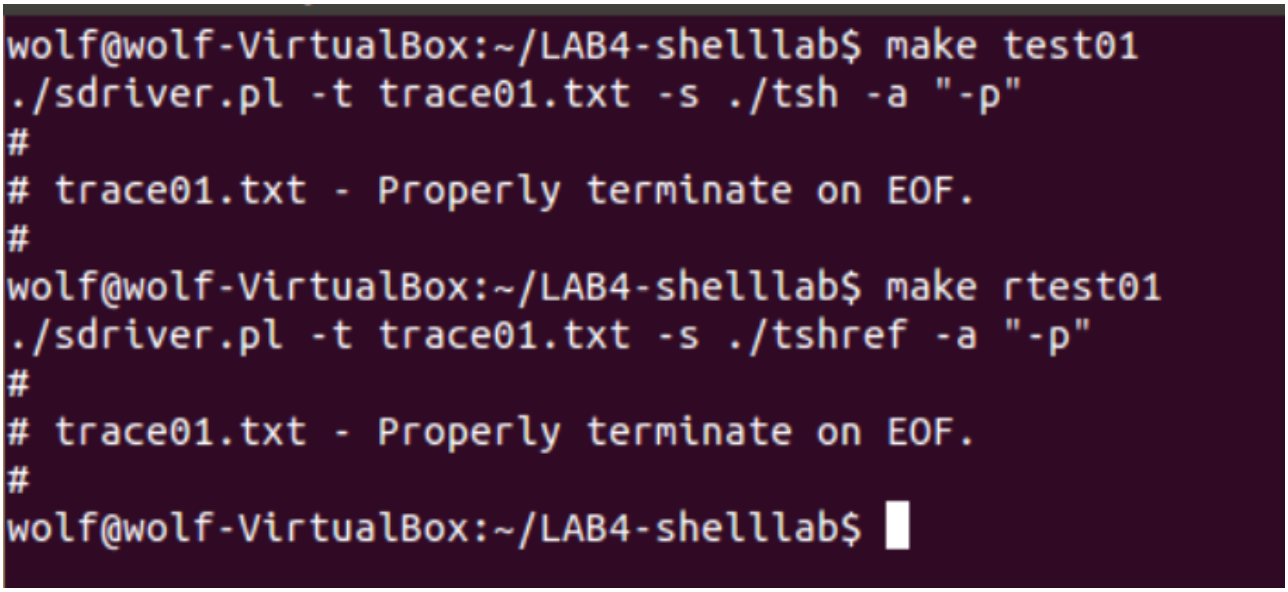
（1）trace01-正确地终止EOF

```
#
# trace01.txt - Properly terminate on EOF.
#
CLOSE
WAIT
```

分析：

调用linux命令close关闭文件并wait等待，在EOF上正常终止。最基础的测试。

运行截图：



（2）trace02-进程内置quit命令

```
#
# trace02.txt - Process builtin quit command.
#
quit
WAIT
```

分析：

针对输入的命令quit退出shell进程，需要解析cmdline，判断若为“quit”字符串则退出。该题需要完成eval函数和builtin_cmd函数。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test02
./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
#
# trace02.txt - Process builtin quit command.
#
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest02
./sdriver.pl -t trace02.txt -s ./tshref -a "-p"
#
# trace02.txt - Process builtin quit command.
#
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（3）trace03-运行一个前台job

```
#
# trace03.txt - Run a foreground job.
#
/bin/echo tsh> quit
quit
```

分析：

eval函数先通过builtin_cmd查询cmdline是不是内置命令如quit，如果是则当前进程执行命令

如果不是则创建一个子进程，在子进程中调用 execve()函数通过 argv[0]来寻找路径，并在子进程中运行路径中的可执行文件，如果找不到则说明命令为无效命令，输出命令无效，并用 exit(0)结束该子进程

/bin/echo就是打开bin目录下的echo文件，echo是一个常见的测试指令，功能是将其后面的内容当作字符串输出。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test03
./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest03
./sdriver.pl -t trace03.txt -s ./tshref -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（4）trace04-运行后台job

```
#
# trace04.txt - Run a background job.
#
/bin/echo -e tsh> ./myspin 1 \046
./myspin 1 &
```

分析：

先在前台执行echo命令，等待程序执行完毕回收子进程。&代表是一个后台程序，myspin睡眠1秒，然后停止。因为成功实现了后台程序的运行，所以会输出后台程序的运行信息。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test04
./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (2694) ./myspin 1 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest04
./sdriver.pl -t trace04.txt -s ./tshref -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (2700) ./myspin 1 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（5）**trace05-**处理**jobs**内置命令

```
#
# trace05.txt - Process jobs builtin command.
#
/bin/echo -e tsh> ./myspin 2 \046
./myspin 2 &

/bin/echo -e tsh> ./myspin 3 \046
./myspin 3 &

/bin/echo tsh> jobs
jobs
```

分析：

分别运行了前台echo、后台myspin、前台echo、后台myspin，然后需要实现一个内置命令job，功能是显示目前任务列表中的所有任务的所有属性。

运行截图：

```
wolf@wolf-VirtualBox:~$ cd LAB4-shelllab/
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make
gcc -Wall -O2    tsh.c   -o tsh
wolf@wolf-VirtualBox:~/LAB4-shelllab$ clear
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test05
./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (2674) ./myspin 2 &
tsh> ./myspin 3 &
[2] (2676) ./myspin 3 &
tsh> jobs
[1] (2674) Running ./myspin 2 &
[2] (2676) Running ./myspin 3 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest05
./sdriver.pl -t trace05.txt -s ./tshref -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (2683) ./myspin 2 &
tsh> ./myspin 3 &
[2] (2685) ./myspin 3 &
tsh> jobs
[1] (2683) Running ./myspin 2 &
[2] (2685) Running ./myspin 3 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（6）trace06-将SIGINT转发到前台作业

```
#
# trace06.txt - Forward SIGINT to foreground job.
#
/bin/echo -e tsh> ./myspin 4
./myspin 4

SLEEP 2
INT
```

分析：

接收到了中断信号SIGINT（即CTRL_C)那么结束前台进程

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test06
./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (2707) terminated by signal 2
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest06
./sdriver.pl -t trace06.txt -s ./tshref -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (2713) terminated by signal 2
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（7）**trace07-仅将SIGINT转发给前台作业**

```
#
# trace07.txt - Forward SIGINT only to foreground job.
#
/bin/echo -e tsh> ./myspin 4 \046
./myspin 4 &

/bin/echo -e tsh> ./myspin 5
./myspin 5

SLEEP 2
INT

/bin/echo tsh> jobs
jobs
```

分析：

只将SIGINT转发给前台作业。测试方法是给出两个作业，一个在前台工作，另一个在后台工作，接下来传递SIGINT指令，然后调用内置指令jobs来查看此时的工作信息，来对比出是不是只将SIGINT转发给前台作业。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test07
./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (2723) ./myspin 4 &
tsh> ./myspin 5
Job [2] (2725) terminated by signal 2
tsh> jobs
[1] (2723) Running ./myspin 4 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest07
./sdriver.pl -t trace07.txt -s ./tshref -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (2732) ./myspin 4 &
tsh> ./myspin 5
Job [2] (2734) terminated by signal 2
tsh> jobs
[1] (2732) Running ./myspin 4 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（8）**trace08-**仅将**SIGTSTP**转发到前台作业

```
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
/bin/echo -e tsh> ./myspin 4 \046
./myspin 4 &

/bin/echo -e tsh> ./myspin 5
./myspin 5

SLEEP 2
TSTP

/bin/echo tsh> jobs
jobs
```

分析：

只将SIGTSTP转发给前台作业。根据这个信号的作用，也就是该进程会停止直到下一个SIGCONT也就是挂起，让别的程序继续运行。这里也就是运行了后台程序，然后使用jobs来打印出进程的信息，进行对比。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test08
./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (2743) ./myspin 4 &
tsh> ./myspin 5
Job [2] (2745) stopped by signal 20
tsh> jobs
[1] (2743) Running ./myspin 4 &
[2] (2745) Stopped ./myspin 5
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest08
./sdriver.pl -t trace08.txt -s ./tshref -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (2752) ./myspin 4 &
tsh> ./myspin 5
Job [2] (2754) stopped by signal 20
tsh> jobs
[1] (2752) Running ./myspin 4 &
[2] (2754) Stopped ./myspin 5
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（9）trace09-进程bg内置命令

```
#
# trace09.txt - Process bg builtin command
#
/bin/echo -e tsh> ./myspin 4 \046
./myspin 4 &

/bin/echo -e tsh> ./myspin 5
./myspin 5

SLEEP 2
TSTP

/bin/echo tsh> jobs
jobs

/bin/echo tsh> bg %2
bg %2
```

```
/bin/echo tsh> jobs
jobs
```

分析：

这个题目的作用是测试bg命令能否让一个被挂起（暂停）的程序（进程2）继续在后台运行。在停止后，先输出进程信息之后，使用bg命令来唤醒进程2，也就是刚才被挂起的程序，接下来继续使用Jobs命令来输出结果。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test09
./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (2763) ./myspin 4 &
tsh> ./myspin 5
Job [2] (2765) stopped by signal 20
tsh> jobs
[1] (2763) Running ./myspin 4 &
[2] (2765) Stopped ./myspin 5
tsh> bg %2
[2] (2765) ./myspin 5
tsh> jobs
[1] (2763) Running ./myspin 4 &
[2] (2765) Running ./myspin 5
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest09
./sdriver.pl -t trace09.txt -s ./tshref -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (2774) ./myspin 4 &
tsh> ./myspin 5
Job [2] (2776) stopped by signal 20
tsh> jobs
[1] (2774) Running ./myspin 4 &
[2] (2776) Stopped ./myspin 5
tsh> bg %2
[2] (2776) ./myspin 5
tsh> jobs
[1] (2774) Running ./myspin 4 &
[2] (2776) Running ./myspin 5
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（10）trace10-进程fg内置命令

```
#
# trace10.txt - Process fg builtin command.
#
/bin/echo -e tsh> ./myspin 4 \046
./myspin 4 &

SLEEP 1
/bin/echo tsh> fg %1
fg %1

SLEEP 1
TSTP

/bin/echo tsh> jobs
jobs

/bin/echo tsh> fg %1
fg %1

/bin/echo tsh> jobs
jobs
```

分析：

测试fg能否让一个被挂起的程序继续在前台运行。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test10
./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (2787) ./myspin 4 &
tsh> fg %1
Job [1] (2787) stopped by signal 20
tsh> jobs
[1] (2787) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest10
./sdriver.pl -t trace10.txt -s ./tshref -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (2797) ./myspin 4 &
tsh> fg %1
Job [1] (2797) stopped by signal 20
tsh> jobs
[1] (2797) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（11）trace11-将SIGINT转发给前台进程组中的每个进程

```
#
# trace11.txt - Forward SIGINT to every process in foreground
process group
#
/bin/echo -e tsh> ./mysplit 4
./mysplit 4

SLEEP 2
INT

/bin/echo tsh> /bin/ps a
/bin/ps a
```

分析：

这里需要将SIGINT发给前台进程组中的每个进程。ps –a 显示所有进程，这里是有两个进程的，mysplit创建了一个子进程，接下来发送指令SIGINT，所以进程组中的所有进程都应该停止，接下来调用ps -a来查看该进程组中的每个进程是否都停止了。

运行截图：

```
#
tsh> ./mysplit 4
Job [1] (2808) terminated by signal 2
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
 1082 tty4     Ss+    0:00 /sbin/getty -8 38400 tty4
 1090 tty5     Ss+    0:00 /sbin/getty -8 38400 tty5
 1096 tty2     Ss+    0:00 /sbin/getty -8 38400 tty2
 1097 tty3     Ss+    0:00 /sbin/getty -8 38400 tty3
 1099 tty6     Ss+    0:00 /sbin/getty -8 38400 tty6
 1140 tty7     Ss+    0:03 /usr/bin/X :0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch -background none
 1836 tty1     Ss+    0:00 /sbin/getty -8 38400 tty1
 2601 pts/0    Ss     0:00 bash
 2803 pts/0    S+     0:00 make test11
 2804 pts/0    S+     0:00 /bin/sh -c ./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
 2805 pts/0    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./tsh -a -p
 2806 pts/0    S+     0:00 ./tsh -p
 2811 pts/0    R      0:00 /bin/ps a
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest11
./sdriver.pl -t trace11.txt -s ./tshref -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (2818) terminated by signal 2
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
 1082 tty4     Ss+    0:00 /sbin/getty -8 38400 tty4
 1090 tty5     Ss+    0:00 /sbin/getty -8 38400 tty5
 1096 tty2     Ss+    0:00 /sbin/getty -8 38400 tty2
 1097 tty3     Ss+    0:00 /sbin/getty -8 38400 tty3
 1099 tty6     Ss+    0:00 /sbin/getty -8 38400 tty6
 1140 tty7     Ss+    0:03 /usr/bin/X :0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch -background none
 1836 tty1     Ss+    0:00 /sbin/getty -8 38400 tty1
 2601 pts/0    Ss     0:00 bash
 2813 pts/0    S+     0:00 make rtest11
 2814 pts/0    S+     0:00 /bin/sh -c ./sdriver.pl -t trace11.txt -s ./tshref -a "-p"
 2815 pts/0    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./tshref -a -p
 2816 pts/0    S+     0:00 ./tshref -p
 2821 pts/0    R      0:00 /bin/ps a
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（12）trace12-将SIGTSTP转发到前台进程组中的每个进程

```
#
# trace12.txt - Forward SIGTSTP to every process in foreground
process group
#
/bin/echo -e tsh> ./mysplit 4
./mysplit 4

SLEEP 2
TSTP

/bin/echo tsh> jobs
jobs

/bin/echo tsh> /bin/ps a
/bin/ps a
```

分析：

测试将SIGTSTP转发给前台进程组中的每个进程，测试方法跟上一题差不多。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test12
./sdriver.pl -t trace12.txt -s ./tsh -a "-p"
#
# trace12.txt - Forward SIGTSTP to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (2828) stopped by signal 20
tsh> jobs
[1] (2828) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
 1082 tty4     Ss+    0:00 /sbin/getty -8 38400 tty4
 1090 tty5     Ss+    0:00 /sbin/getty -8 38400 tty5
 1096 tty2     Ss+    0:00 /sbin/getty -8 38400 tty2
 1097 tty3     Ss+    0:00 /sbin/getty -8 38400 tty3
 1099 tty6     Ss+    0:00 /sbin/getty -8 38400 tty6
 1140 tty7     Ss+    0:03 /usr/bin/X :0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch -background none
 1836 tty1     Ss+    0:00 /sbin/getty -8 38400 tty1
 2601 pts/0    Ss     0:00 bash
 2823 pts/0    S+     0:00 make test12
 2824 pts/0    S+     0:00 /bin/sh -c ./sdriver.pl -t trace12.txt -s ./tsh -a "-p"
 2825 pts/0    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace12.txt -s ./tsh -a -p
 2826 pts/0    S+     0:00 ./tsh -p
 2828 pts/0    T      0:00 ./mysplit 4
 2829 pts/0    T      0:00 ./mysplit 4
 2832 pts/0    R      0:00 /bin/ps a
```

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest12
./sdriver.pl -t trace12.txt -s ./tshref -a "-p"
#
# trace12.txt - Forward SIGTSTP to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (2838) stopped by signal 20
tsh> jobs
[1] (2838) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
 1082 tty4     Ss+    0:00 /sbin/getty -8 38400 tty4
 1090 tty5     Ss+    0:00 /sbin/getty -8 38400 tty5
 1096 tty2     Ss+    0:00 /sbin/getty -8 38400 tty2
 1097 tty3     Ss+    0:00 /sbin/getty -8 38400 tty3
 1099 tty6     Ss+    0:00 /sbin/getty -8 38400 tty6
 1140 tty7     Ss+    0:03 /usr/bin/X :0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch -background none
 1836 tty1     Ss+    0:00 /sbin/getty -8 38400 tty1
 2601 pts/0    Ss     0:00 bash
 2833 pts/0    S+     0:00 make rtest12
 2834 pts/0    S+     0:00 /bin/sh -c ./sdriver.pl -t trace12.txt -s ./tshref -a "-p"
 2835 pts/0    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace12.txt -s ./tshref -a -p
 2836 pts/0    S+     0:00 ./tshref -p
 2838 pts/0    T      0:00 ./mysplit 4
 2839 pts/0    T      0:00 ./mysplit 4
 2842 pts/0    R      0:00 /bin/ps a
```

（13）trace13-重新启动进程组中的每个已停止的进程

```
#
# trace13.txt - Restart every stopped process in process group
#
/bin/echo -e tsh> ./mysplit 4
./mysplit 4

SLEEP 2
TSTP

/bin/echo tsh> jobs
```

```
jobs

/bin/echo tsh> /bin/ps a
/bin/ps a

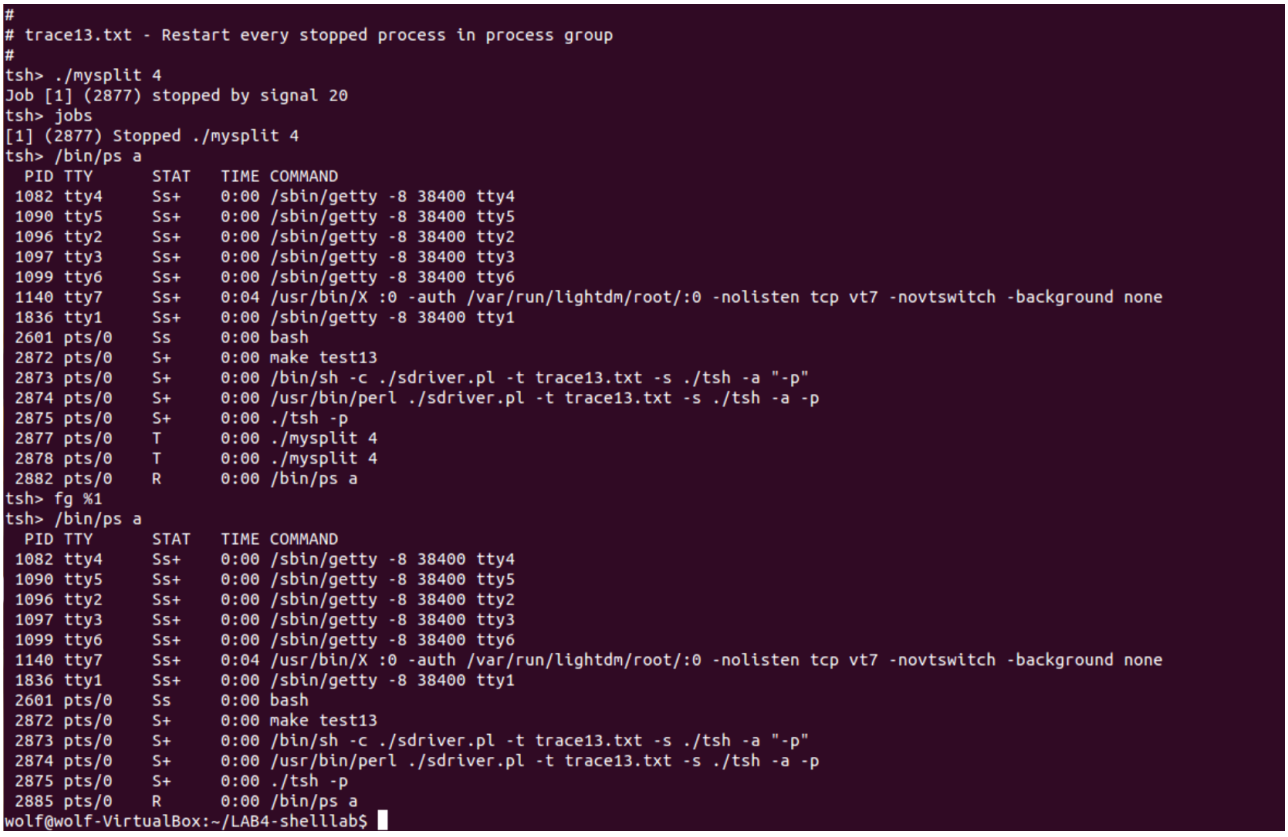
/bin/echo tsh> fg %1
fg %1

/bin/echo tsh> /bin/ps a
/bin/ps a
```

分析：

测试fg是否能唤醒整个工作，使用ps -a来查看停止整个工作和唤醒整个工作的区别

运行截图：



```
#
# trace13.txt - Restart every stopped process in process group
#
tsh> ./mysplit 4
Job [1] (2891) stopped by signal 20
tsh> jobs
[1] (2891) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
 1082 tty4     Ss+    0:00 /sbin/getty -8 38400 tty4
 1090 tty5     Ss+    0:00 /sbin/getty -8 38400 tty5
 1096 tty2     Ss+    0:00 /sbin/getty -8 38400 tty2
 1097 tty3     Ss+    0:00 /sbin/getty -8 38400 tty3
 1099 tty6     Ss+    0:00 /sbin/getty -8 38400 tty6
 1140 tty7     Ss+    0:04 /usr/bin/X :0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch -background none
 1836 tty1     Ss+    0:00 /sbin/getty -8 38400 tty1
 2601 pts/0    Ss     0:00 bash
 2886 pts/0    S+     0:00 make rtest13
 2887 pts/0    S+     0:00 /bin/sh -c ./sdriver.pl -t trace13.txt -s ./tshref -a "-p"
 2888 pts/0    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tshref -a -p
 2889 pts/0    S+     0:00 ./tshref -p
 2891 pts/0    T      0:00 ./mysplit 4
 2892 pts/0    T      0:00 ./mysplit 4
 2895 pts/0    R      0:00 /bin/ps a
tsh> fg %1
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
 1082 tty4     Ss+    0:00 /sbin/getty -8 38400 tty4
 1090 tty5     Ss+    0:00 /sbin/getty -8 38400 tty5
 1096 tty2     Ss+    0:00 /sbin/getty -8 38400 tty2
 1097 tty3     Ss+    0:00 /sbin/getty -8 38400 tty3
 1099 tty6     Ss+    0:00 /sbin/getty -8 38400 tty6
 1140 tty7     Ss+    0:04 /usr/bin/X :0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch -background none
 1836 tty1     Ss+    0:00 /sbin/getty -8 38400 tty1
 2601 pts/0    Ss     0:00 bash
 2886 pts/0    S+     0:00 make rtest13
 2887 pts/0    S+     0:00 /bin/sh -c ./sdriver.pl -t trace13.txt -s ./tshref -a "-p"
 2888 pts/0    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tshref -a -p
 2889 pts/0    S+     0:00 ./tshref -p
 2898 pts/0    R      0:00 /bin/ps a
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（14）**trace14-**简单的错误处理

```
#
# trace14.txt - Simple error handling
#
/bin/echo tsh> ./bogus
./bogus

/bin/echo -e tsh> ./myspin 4 \046
./myspin 4 &

/bin/echo tsh> fg
fg

/bin/echo tsh> bg
bg

/bin/echo tsh> fg a
fg a

/bin/echo tsh> bg a
bg a
```

```
/bin/echo tsh> fg 9999999
fg 9999999

/bin/echo tsh> bg 9999999
bg 9999999

/bin/echo tsh> fg %2
fg %2

/bin/echo tsh> fg %1
fg %1

SLEEP 2
TSTP

/bin/echo tsh> bg %2
bg %2

/bin/echo tsh> bg %1
bg %1

/bin/echo tsh> jobs
jobs
```

分析：

检测错误处理系统是否正确。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test14
./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (2906) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 9999999
(9999999): No such process
tsh> bg 9999999
(9999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (2906) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (2906) ./myspin 4 &
tsh> jobs
[1] (2906) Running ./myspin 4 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest14
./sdriver.pl -t trace14.txt -s ./tshref -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (2946) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 9999999
(9999999): No such process
tsh> bg 9999999
(9999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (2946) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (2946) ./myspin 4 &
tsh> jobs
[1] (2946) Running ./myspin 4 &
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（15）trace15-综合测试

```
#
# trace15.txt - Putting it all together
#

/bin/echo tsh> ./bogus
./bogus

/bin/echo tsh> ./myspin 10
./myspin 10

SLEEP 2
INT

/bin/echo -e tsh> ./myspin 3 \046
./myspin 3 &

/bin/echo -e tsh> ./myspin 4 \046
./myspin 4 &

/bin/echo tsh> jobs
jobs

/bin/echo tsh> fg %1
fg %1

SLEEP 2
TSTP

/bin/echo tsh> jobs
jobs

/bin/echo tsh> bg %3
bg %3

/bin/echo tsh> bg %1
bg %1

/bin/echo tsh> jobs
jobs
```

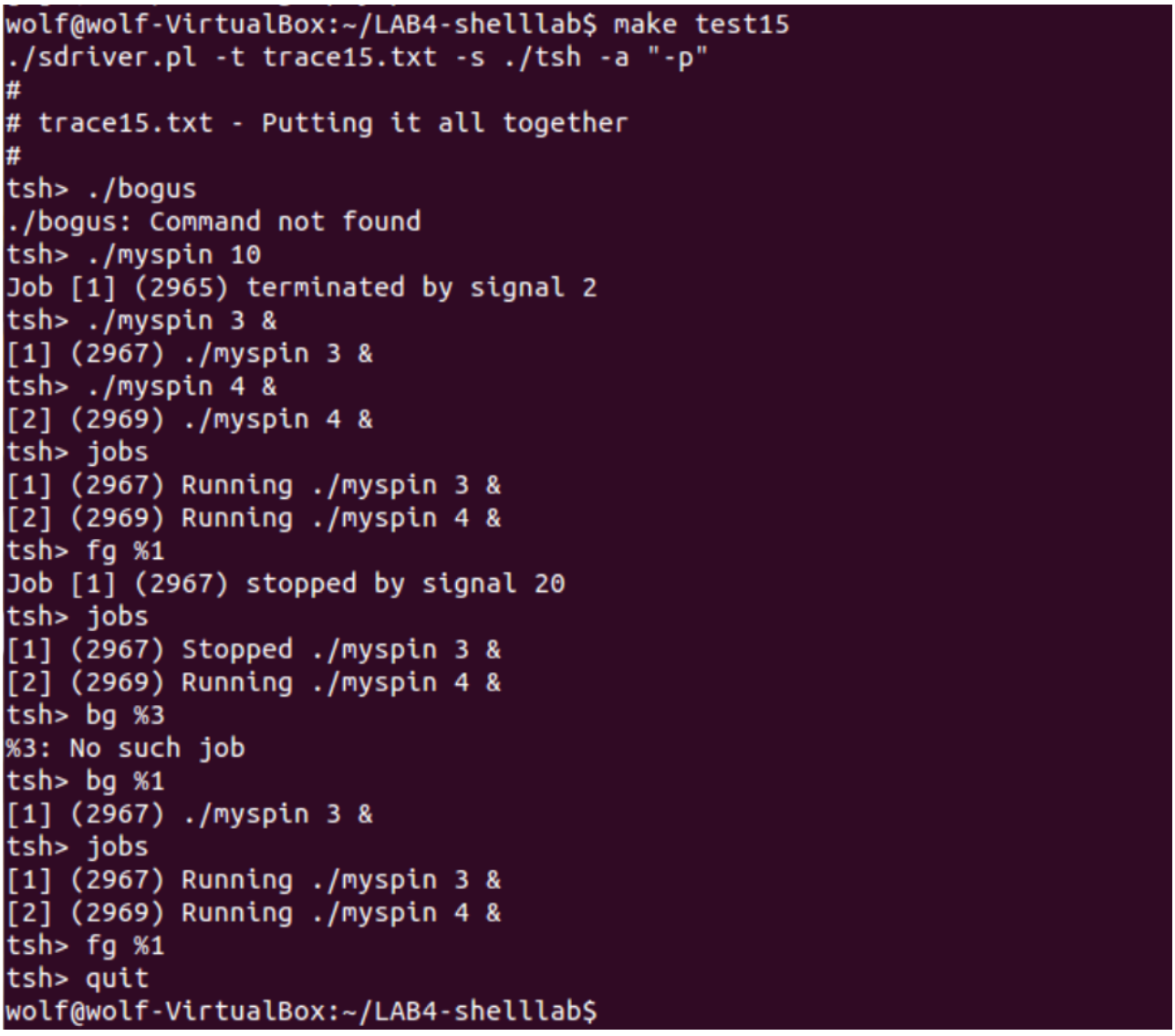
```
/bin/echo tsh> fg %1
fg %1

/bin/echo tsh> quit
quit
```

分析：

综合测试。

运行截图：



```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest15
./sdriver.pl -t trace15.txt -s ./tshref -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 10
Job [1] (2986) terminated by signal 2
tsh> ./myspin 3 &
[1] (2988) ./myspin 3 &
tsh> ./myspin 4 &
[2] (2990) ./myspin 4 &
tsh> jobs
[1] (2988) Running ./myspin 3 &
[2] (2990) Running ./myspin 4 &
tsh> fg %1
Job [1] (2988) stopped by signal 20
tsh> jobs
[1] (2988) Stopped ./myspin 3 &
[2] (2990) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (2988) ./myspin 3 &
tsh> jobs
[1] (2988) Running ./myspin 3 &
[2] (2990) Running ./myspin 4 &
tsh> fg %1
tsh> quit
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

（16）**trace16-**来自其他进程的信号

```
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and
SIGINT
#     signals that come from other processes instead of the
terminal.
#

/bin/echo tsh> ./mystop 2
./mystop 2

SLEEP 3

/bin/echo tsh> jobs
jobs

/bin/echo tsh> ./myint 2
./myint 2
```

分析：

测试shell是否可以处理来自其他进程而不是终端的SIGTSTP和SIGINT信号。

运行截图：

```
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make test16
./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#     signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (3004) stopped by signal 20
tsh> jobs
[1] (3004) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (3007) terminated by signal 2
wolf@wolf-VirtualBox:~/LAB4-shelllab$ make rtest16
./sdriver.pl -t trace16.txt -s ./tshref -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#     signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (3014) stopped by signal 20
tsh> jobs
[1] (3014) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (3017) terminated by signal 2
wolf@wolf-VirtualBox:~/LAB4-shelllab$
```

★2 自动验证

我们可以修改makefile，自己给它新增一个功能，使它能够一次性打印所有的测试点数据。跟参考输出的比对可以使用meld工具进行进行。

（1）修改**makefile**文件

在原makefile文件下方加入以下代码

（本质是把之前单次运行的指令给一次性输出了）

```
testall:
	$(DRIVER) -t trace01.txt -s $(TSH) -a $(TSHARGS)
	$(DRIVER) -t trace02.txt -s $(TSH) -a $(TSHARGS)
	$(DRIVER) -t trace03.txt -s $(TSH) -a $(TSHARGS)
	$(DRIVER) -t trace04.txt -s $(TSH) -a $(TSHARGS)
	$(DRIVER) -t trace05.txt -s $(TSH) -a $(TSHARGS)
	$(DRIVER) -t trace06.txt -s $(TSH) -a $(TSHARGS)
	$(DRIVER) -t trace07.txt -s $(TSH) -a $(TSHARGS)
```

```
    $(DRIVER) -t trace08.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace09.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace10.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace11.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace12.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace13.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace14.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace15.txt -s $(TSH) -a $(TSHARGS)
    $(DRIVER) -t trace16.txt -s $(TSH) -a $(TSHARGS)

rtestall:
    $(DRIVER) -t trace01.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace02.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace03.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace04.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace05.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace06.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace07.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace08.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace09.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace10.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace11.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace12.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace13.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace14.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace15.txt -s $(TSHREF) -a $(TSHARGS)
    $(DRIVER) -t trace16.txt -s $(TSHREF) -a $(TSHARGS)
```

（2）测试所有测试集并使用重定向符将其导出

★请等待一条指令运行完成后再执行下一条指令

★特别不要开两个终端同时并行操作，因为执行测试机时涉及到运行`ps -a`，如果两个同时运行，会将另一个终端也显示进去，导致my_output和reference_output差距很大。

在终端依次输入以下指令

```
make testall > my_output.txt
make rtestall > reference_output.txt
```

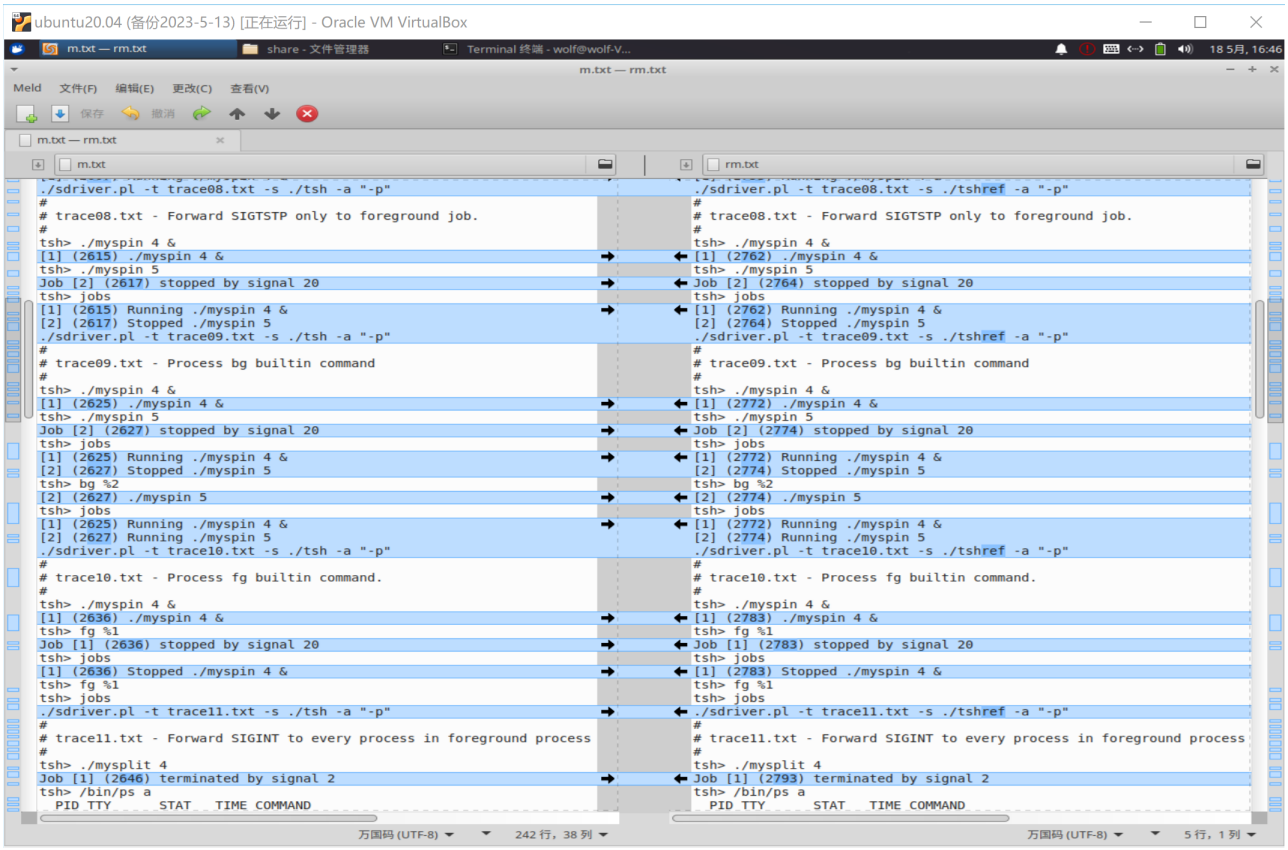
从my_output.txt和reference_output.txt分别拿到结果。

（3）使用**meld**进行比对

这里使用的是ubuntu20.04，因为我们原本的ubuntu14.04太古老了，找不到meld包，没法安装meld。

通过meld比对可以发现，我们的和参考的只有进程编号上有区别，其余都是一致的。

进程编号上的区别是必然存在的，因为操作系统是Linux操作系统，我们依次生成的实际上都是子进程。这些在Linux下生成的子进程都对应着自己的PID，这是独有的，所以会不一样。



通过比对可以确认实验的结果基本正确。

★3 实验总结

进程的创建与销毁都是内核级的操作，这些操作由操作系统完成，一般来说用户是没有权限看到的。用户能做的只是请求创建进程、销毁进程、中断、结束等。Shell俗称壳（用来区别于核），是指“为使用者提供操作界面”的软件，它的作用是沟通用户与内核。我们的tsh（也就是tinyshell）所做的事情就是Shell的简化版，它简化到只有4个信号（SIGINT，SIGTSTP,SIGCHLD,SIGCONT），只有4个内置命令（bg,fg,jobs,quit）。它起到的作用也是

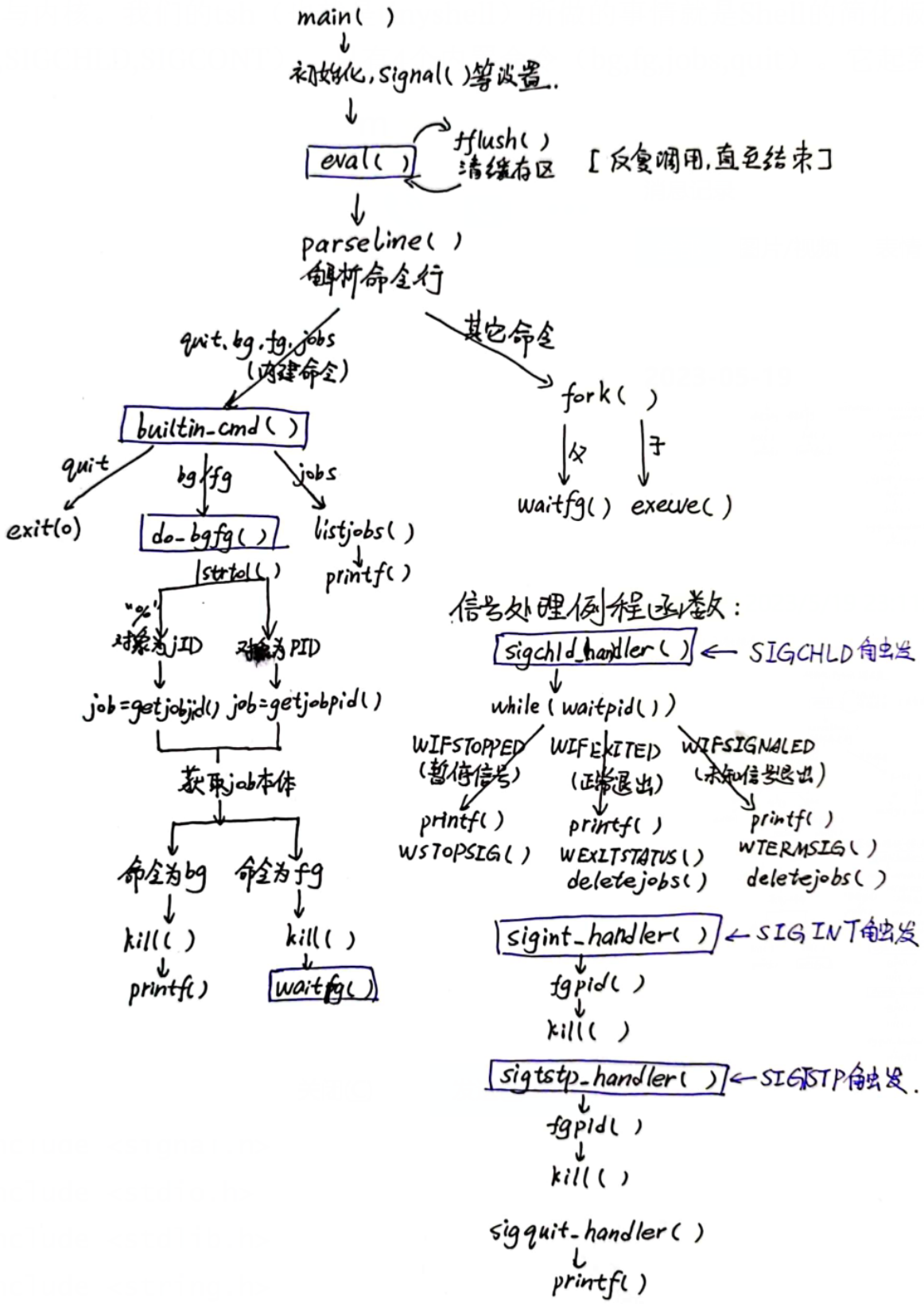
沟通我们用户与内核，满足我们用户对于操作系统的要求，包括谁在前台运行，运行什么，谁在后台运行，运行什么等。

实际上，我认为tinyshell有两个视角去观察它。

★1 函数视角

其中带方框的为我们需要实现的部分。

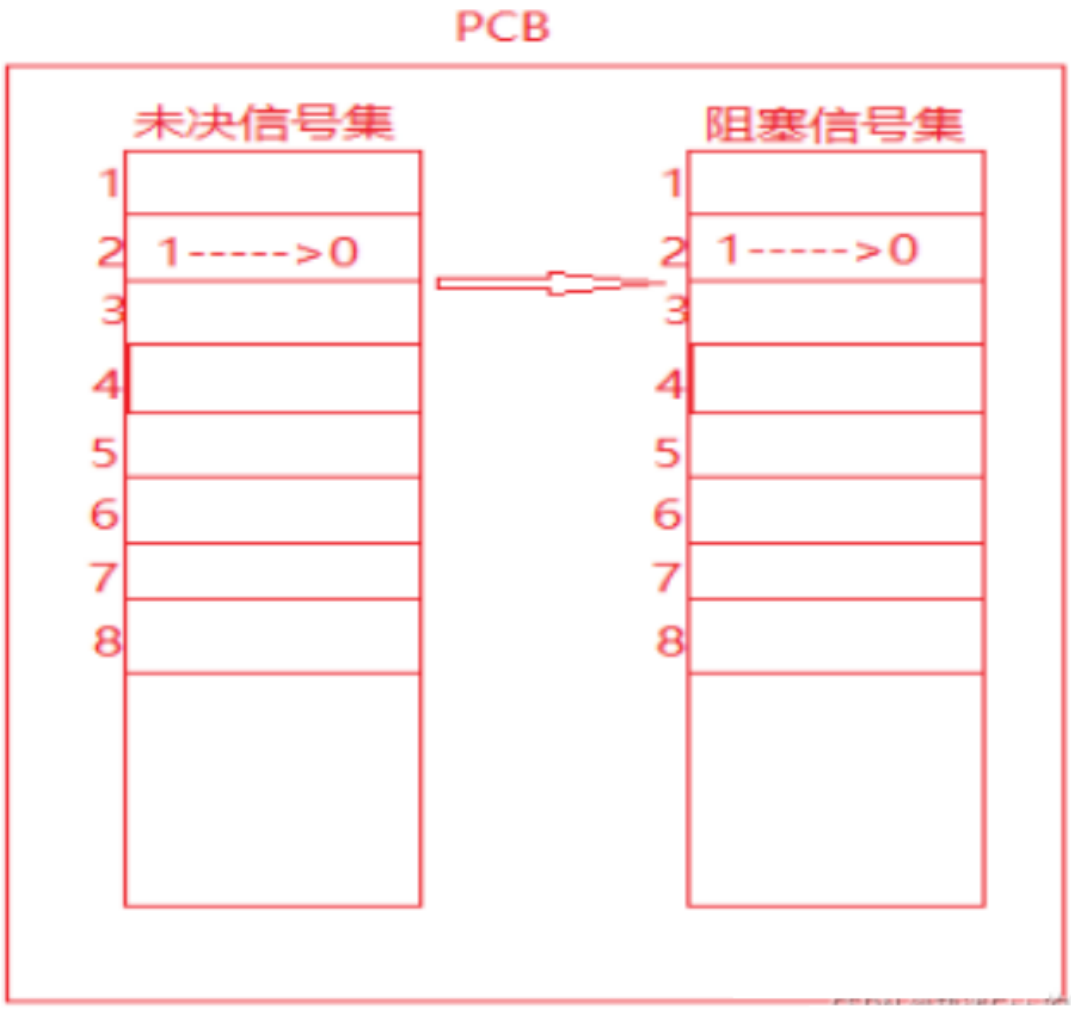
(1) 函数视角



从函数的视角理解，可以让我们将不同的函数与过程之间互相调用的关系看的更加透彻，能知道什么时候干什么事情。

★2 信号视角

每个进程都维护着两个集合。称为阻塞信号集（也称作屏蔽词集）和未决信号集。



阻塞信号集是当前进程要阻塞的信号的集合，未决信号集是当前进程中还处于未决状态的信号的集合，这两个集合存储在内核的PCB中。

> 下面以*SIGINT*为例说明信号未决信号集和阻塞信号集的关系：
>
> 当进程收到一个*SIGINT*信号（信号编号为*2*），首先这个信号会保存在未决信号集合中，此时对应的*2*号编号的这个位置上置为*1*，表示处于未决状态；在这个信号需要被处理之前首先要在阻塞信号集中的编号为*2*的位置上去检查该值是否为*1*：
>
> 如果为*1*，表示*SIGNIT*信号被当前进程阻塞了，这个信号暂时不被处理，所以未决信号集 上该位置上的值保持为*1*，表示该信号处于未决状态；
>
> 如果为*0*，表示*SIGINT*信号没有被当前进程阻塞，这个信号需要被处理，内核会对*SIGINT*信号进行处理（执行默认动作，忽略或者执行用户自定义的信号处理函数），并将未决信号集中编号为*2*的位置上将*1*变为*0*，表示该信号已经处理了，这个时间非常短暂，用户感知不到。
>
> 当*SIGINT*信号从阻塞信号集中解除阻塞之后，该信号就会被处理。

事实上，main函数一开始就调用如下的signal函数，这些函数的作用是，一旦进程接收到这4个信号，就立刻调用相应的处理例程。实际上，其中的三个就是我们要实现的。

```
    Signal(SIGINT, sigint_handler);   /* ctrl-c */
    Signal(SIGTSTP, sigtstp_handler); /* ctrl-z */
    Signal(SIGCHLD, sigchld_handler); /* Terminated or stopped
child */
    Signal(SIGQUIT, sigquit_handler);
```

然后，在函数的实现中，信号相关的函数也如影随形。

我们遇到了如下的信号设置函数，它们能够原子性地执行设置或修改信号集或者信号。

```
sigemptyset:     int sigemptyset(sigset_t *set);
sigaddset:       int sigaddset(sigset_t *set,int signum);
sigprocmask:     int sigprocmask(int how, const sigset_t *restrict
set, sigset_t *restrict oldset);
sigsuspend:      int sigsuspend(const sigset_t *mask);
```

特别地，关注到对于信号的管理更倾向于使用set来进行整合与打包，将几个信号放入信号集里面一起操作。

比如void waitfg(pid_t pid)函数中的sigsuspend就可以原子性地将我们自定义的wait空信号集替换本进程的原有阻塞信号集，从而使得该进程可以被任何信号唤醒。

从信号的视角理解，可以让我们对于不同进程之间的通信有更深入的洞察。

★并发问题

这里存在两个并发问题

（1）jobs数组是一个临界缓冲区，addjob函数和deletejob函数的访问需要持有锁。

（2）sigchld处理函数处理SIGCHLD信号的时候，同时至多处理两个子进程的删除。此时需要用while信号处理同时并发的多个子进程

4 完整代码

在完整代码中加入了自己的注释与理解。

```
/*
 * tsh - A tiny shell program with job control
 * 202108010206 ArcticWolf
 */
#include <ctype.h>
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

/* Misc manifest constants */
#define MAXLINE 1024   /* max line size */
#define MAXARGS 128    /* max args on a command line */
#define MAXJOBS 16     /* max jobs at any point in time */
#define MAXJID 1 << 16 /* max job ID */

/* Job states */
#define UNDEF 0 /* undefined */
#define FG 1    /* running in foreground */
#define BG 2    /* running in background */
#define ST 3    /* stopped */

/*
 * Jobs states: FG (foreground), BG (background), ST (stopped)
 * Job state transitions and enabling actions:
 *     FG -> ST  : ctrl-z
 *     ST -> FG  : fg command
 *     ST -> BG  : bg command
 *     BG -> FG  : fg command
 * At most 1 job can be in the FG state.
 */

/* Global variables */
extern char **environ;   /* defined in libc */
```

```
char prompt[] = "tsh> "; /*命令行提示符(DO NOT CHANGE) */
int verbose = 0;         /*是否输出额外信息*/
int nextjid = 1;         /*要分配的下一个作业ID*/
char sbuf[MAXLINE];      /* for composing sprintf messages */

struct job_t
{                          /* The job struct */
    pid_t pid;             /* job PID */
    int jid;               /* job ID [1, 2, ...] */
    int state;             /* UNDEF, BG, FG, or ST */
    char cmdline[MAXLINE]; /* command line */
};
struct job_t jobs[MAXJOBS]; /* The job list */
/* End global variables */

/* Function prototypes */

/* Here are the functions that you will implement */
void eval(char *cmdline);
int builtin_cmd(char **argv);
void do_bgfg(char **argv);
void waitfg(pid_t pid);

void sigchld_handler(int sig);
void sigtstp_handler(int sig);
void sigint_handler(int sig);

/* Here are helper routines that we've provided for you */
int parseline(const char *cmdline, char **argv);
void sigquit_handler(int sig);

void clearjob(struct job_t *job);
void initjobs(struct job_t *jobs);
int maxjid(struct job_t *jobs);
int addjob(struct job_t *jobs, pid_t pid, int state, char
*cmdline);
int deletejob(struct job_t *jobs, pid_t pid);
pid_t fgpid(struct job_t *jobs);
struct job_t *getjobpid(struct job_t *jobs, pid_t pid);
struct job_t *getjobjid(struct job_t *jobs, int jid);
int pid2jid(pid_t pid);
void listjobs(struct job_t *jobs);
```

```c
void usage(void);
void unix_error(char *msg);
void app_error(char *msg);
typedef void handler_t(int);
handler_t *Signal(int signum, handler_t *handler);

/*
 * main - shell的主函数
 */
int main(int argc, char **argv)
{
    char c;
    char cmdline[MAXLINE];
    int emit_prompt = 1; /* emit prompt (default) */

    /* 将stderr重定向到stdout（这样驱动程序将获得连接到stdout的管道上的所有输出） */
    dup2(1, 2);

    /* 解析命令行 */
    while ((c = getopt(argc, argv, "hvp")) != EOF)
    {
        switch (c)
        {
        case 'h': /* 打印提示信息 */
            usage();
            break;
        case 'v': /* 发出附加诊断信息 */
            verbose = 1;
            break;
        case 'p':            /* 不打印提示 */
            emit_prompt = 0; /* 便于自动测试 */
            break;
        default:
            usage();
        }
    }

    /* Install the signal handlers */

    /* These are the ones you will need to implement */
```

```
    Signal(SIGINT, sigint_handler);   /* ctrl-c */
    Signal(SIGTSTP, sigtstp_handler); /* ctrl-z */
    Signal(SIGCHLD, sigchld_handler); /* Terminated or stopped
child */

    /* This one provides a clean way to kill the shell */
    Signal(SIGQUIT, sigquit_handler);

    /* 初始化 job list */
    initjobs(jobs);

    /* Execute the shell's read/eval loop */
    while (1)
    {

        /* Read command line */
        if (emit_prompt)
        {
            printf("%s", prompt);
            fflush(stdout); // 清空缓存区
        }
        if ((fgets(cmdline, MAXLINE, stdin) == NULL) &&
ferror(stdin))
            app_error("fgets error");
        if (feof(stdin))
        { /* End of file (ctrl-d) */
            fflush(stdout);
            exit(0);
        }

        /* Evaluate the command line */
        eval(cmdline);
        fflush(stdout);
        fflush(stdout);
    }

    exit(0); /* control never reaches here */
}

/*
* eval - 分析命令，并派生子进程执行 主要功能是解析cmdline并运行
```

```
如果用户请求了一个内置命令（quit、jobs、bg或fg），则立即执行。否则，派生一个子进程并在该子进程的上下文中运行作业。如果作业正在前台运行，请等待它终止，然后返回。注意：每个子进程都必须有一个唯一的进程组ID，这样当我们在键盘上键入ctrl-c（ctrl-z）时，我们的后台子进程就不会从内核接收SIGINT（SIGTSTP）。
*/
void eval(char *cmdline)
{
    char *argv[MAXARGS]; // execve()函数的参数
    int state = UNDEF;   // 工作状态，FG或BG
    sigset_t set;
    pid_t pid; // 进程id

    // 处理输入的数据
    if (parseline(cmdline, argv) == 1) // 解析命令行，返回给argv数组
        state = BG;
    else
        state = FG;
    if (argv[0] == NULL) // 若命令行为空，parseline也会返回1，但对argv[0]判定后，eval在这里直接返回
        return;

    // 如果不是内置命令
    if (!builtin_cmd(argv)) // 若是内置命令，builtin_cmd(argv)会执行，若该函数返回0，则表示非内置命令
    {
        // 初始化信号集set并把SIGINT SIGTSTP SIGCHLD三个信号放入信号集中，方便管理
        if (sigemptyset(&set) < 0)
            unix_error("sigemptyset error");
        if (sigaddset(&set, SIGINT) < 0 || sigaddset(&set, SIGTSTP) < 0 || sigaddset(&set, SIGCHLD) < 0)
            unix_error("sigaddset error");

        // 阻塞SIGCHLD信号，具体原因在书P541有解释
        // 主要是防止fork之后调度执行子进程并在addjob之前结束子进程，
        // 此时SIGCHLD信号使父进程将子进程回收，这会导致 addjob 和 deletejob 函数执行错位
        // 结果是删除一个不存在的进程号，添加一个不存在且永不会被删除的进程号
        if (sigprocmask(SIG_BLOCK, &set, NULL) < 0)
            unix_error("sigprocmask error");

        if ((pid = fork()) < 0) // fork创建子进程失败
```

```
            unix_error("fork error");
        else if (pid == 0) // fork创建子进程
        {
            // 子进程控制流从这里开始

            if (sigprocmask(SIG_UNBLOCK, &set, NULL) < 0) // 解除阻塞
                unix_error("sigprocmask error");

            // 函数原型 int setpgid(pid_t pid,pid_t pgid);
            // 将pid进程的进程组ID设置成pgid
            // 如果参数pid为0，则会用来设置该进程的组识别码，
            // 如果参数pgid为0，则由pid指定的进程ID将用作进程组ID
            // 一个进程只能为它自己或它的子进程设置进程组ID,不能为其父进程设置
ID。
            if (setpgid(0, 0) < 0) // 设置子进程id（实际上并没有分进程组，
因为一人一组）
                unix_error("setpgid error");

            if (execve(argv[0], argv, environ) < 0)
            {
                printf("%s: Command not found\n", argv[0]);
                exit(0);
            }
        }

        // 父进程控制流从这里开始
        addjob(jobs, pid, state, cmdline); // 将当前进程添加进jobs中，
参数为当前进程pid,state,cmdline

        // 恢复受阻塞的信号 SIGINT SIGTSTP SIGCHLD
        if (sigprocmask(SIG_UNBLOCK, &set, NULL) < 0)
            unix_error("sigprocmask error");

        // 判断子进程类型并做处理
        if (state == FG)
            waitfg(pid); // 等待子进程的前台作业完成
        else
            printf("[%d] (%d) %s", pid2jid(pid), pid, cmdline); //
将进程id映射到job id
    }
    return;
}
```

```c
/*
 * parceline - 分析命令行并构建argv数组
单引号中的字符被视为一个参数。如果用户已请求BG作业，则返回true；如果用户已申请FG作业，则为false
 */
int parseline(const char *cmdline, char **argv)
{
    static char array[MAXLINE]; /* 创建cmd的备份 */
    char *buf = array;          /* 遍历命令行的ptr */
    char *delim;                /* 迭代指针（指向当前处理位置） */
    int argc;                   /* argv向量个数 */
    int bg;                     /* 是否后台运行标志 */

    strcpy(buf, cmdline);          // 在本地生成cmdline的副本buf
    buf[strlen(buf) - 1] = ' ';    /*将cmdline最后的'\n'替换为空格符*/
    while (*buf && (*buf == ' ')) /*忽略前导空格*/
        buf++;

    /*生成参数向量（agrv）列表*/
    argc = 0;
    if (*buf == '\'')
    {
        buf++;
        delim = strchr(buf, '\'');
    }
    else
    {
        delim = strchr(buf, ' ');
    }

    while (delim)
    {
        argv[argc++] = buf;
        *delim = '\0';
        buf = delim + 1;
        while (*buf && (*buf == ' ')) /*忽略空格*/
            buf++;

        if (*buf == '\'')
        {
            buf++;
```

```
            delim = strchr(buf, '\'');
        }
        else
        {
            delim = strchr(buf, ' ');
        }
    }
    argv[argc] = NULL;

    if (argc == 0) /*若cmdline是空的，返回1*/
        return 1;

    /*是否后台运行*/
    if ((bg = (*argv[argc - 1] == '&')) != 0)
    {
        argv[--argc] = NULL; // 去掉&
    }
    return bg; // 若最后有&，则bg=1，parseline会返回1
}

/*
 * builtin_cmd：解析和执行bulidin命令，包括 quit, fg, bg, jobs
 */
int builtin_cmd(char **argv)
{
    if (!strcmp(argv[0], "quit")) // 如果命令是quit，退出
        exit(0);
    else if (!strcmp(argv[0], "bg") || !strcmp(argv[0], "fg")) //
如果是bg或者fg命令，执行do_fgbg函数
        do_bgfg(argv);
    else if (!strcmp(argv[0], "jobs")) // 如果命令是jobs，列出正在运行和
停止的后台作业
        listjobs(jobs);
    else
        return 0; /* 不是内置命令，以0返回eval */
    return 1;
}

/*
 * do_bgfg - 执行bg和fg命令
 */
void do_bgfg(char **argv)
```

```
{
    int num;
    struct job_t *job;
    // 没有参数的fg/bg不符合规定
    if (!argv[1])
    { // 命令行为空
        printf("%s command requires PID or %%jobid argument\n",
argv[0]);
        return;
    }

    // strtol函数原型：long int strtol(const char *nptr, char
**endptr, int base);
    // strtol函数会将参数nptr字符串根据参数base来转换成长整型数，参数base范围
从2至36。

    // 检测fg/bg参数，其中%开头的数字是JobID，纯数字的是PID
    // 找到jobID或PID后通过这个找出job
    if (argv[1][0] == '%')
    {                                                   // 解析jid
        if ((num = strtol(&argv[1][1], NULL, 10)) <= 0) // 获取jid
        {
            printf("%s: argument must be a PID or %%jobid\n",
argv[0]); // 失败,打印错误消息
            return;
        }
        if ((job = getjobjid(jobs, num)) == NULL) // 根据jid获取job
        {
            printf("%%%d: No such job\n", num); // 没找到对应的job
            return;
        }
    }
    else
    {                                                 // 解析PID
        if ((num = strtol(argv[1], NULL, 10)) <= 0) // 获取PID
        {
            printf("%s: argument must be a PID or %%jobid\n",
argv[0]); // 失败,打印错误消息
            return;
        }
        if ((job = getjobpid(jobs, num)) == NULL) // 根据PID获取job
        {
```

```
            printf("(%d): No such process\n", num); // 没找到对应的进
程
            return;
        }
    }

    // kill函数原型： int kill(pid_t pid,int signo)
    // pid > 0：将信号发送给进程 ID 为 pid 的进程。
    // pid ==0：将信号发送给与发送进程属于同一进程组的所有进程。
    // pid < 0：将信号发送给进程组 ID 等于 pid 的绝对值的所有进程。
    // pid ==-1：将信号发送给系统中所有进程。
    if (!strcmp(argv[0], "bg")) // 该进程需要在后台运行
    {
        // bg会启动子进程，并将其放置于后台执行
        job->state = BG;                    // 设置状态BG
        if (kill(-job->pid, SIGCONT) < 0) // 采用负数发送信号到进程组
            unix_error("kill error");
        printf("[%d] (%d) %s", job->jid, job->pid, job->cmdline);
    }
    else if (!strcmp(argv[0], "fg")) // 该进程需要在前台运行
    {
        job->state = FG;                    // 设置状态FG
        if (kill(-job->pid, SIGCONT) < 0) // 采用负数发送信号到进程组
            unix_error("kill error");
        // 当一个进程被设置为前台执行时，当前tsh应该等待该子进程结束
        waitfg(job->pid);
    }
    else // 指令出现异常
    {
        puts("do_bgfg: Internal error");
        exit(0);
    }
    return;
}

/*
 * waitfg - Block until process pid is no longer the foreground
process
 */
void waitfg(pid_t pid)
{
    // 通过pid获取该pid对应的job本体
```

```
    struct job_t *job = getjobpid(jobs, pid);
    if (!job)
        return;

    // 新设立一个wait信号集
    sigset_t wait;
    if (sigemptyset(&wait) < 0) // 清空wait信号集，该信号集为空
        unix_error("sigemptyset &wait error");

    // sigsuspend函数原型： int sigsuspend(const sigset_t *mask);
    // 用于在接收到某个信号之前，临时用mask替换进程的信号掩码，并暂停进程执行，直到收到信号为止

    //   如果当前子进程的状态没有发生改变，则tsh继续休眠
    while (job->state == FG)
        // 一旦有信号改变，就判定是否
        sigsuspend(&wait); // 使用空信号集替换信号掩码，即信号掩码为空，此时任何信号都会唤醒该进程

    return;
}

/*****************
 * Signal handlers
 *****************/

/*
 * sigchld_handler- 处理僵尸子进程例程
 每当子作业终止（变成僵尸），或者因为接收到SIGSTOP或SIGTSTP信号而停止时，内核都会向shell发送sigchld。处理程序获取所有可用的僵尸子进程，但不等待任何其他当前正在运行的子进程终止。
 */
void sigchld_handler(int sig)
{
    int status, jid;
    pid_t pid;
    struct job_t *job;

    if (verbose)
        puts("sigchld_handler: entering"); // 输出额外信息

    /*
```

```
    waitpid函数原型： pid_t waitpid(pid_t pid , int *status , int
options)
    总体：
    如果没有子进程或其它错误原因，则返回-1；
    如果成功回收子进程，则返回回收的那个子进程的ID；
    如果第三个参数为WNOHANG，且子进程都在运行，则返回0
    参数：
    pid：从参数的名字上可以看出来这是一个进程的ID。但是这里pid的值不同时，会有不
同的意义。
        1.pid > 0时，只等待进程ID等于pid的子进程，只要该子进程不结束，就会一直
等待下去；
        2.pid = -1时，等待任何一个子进程的退出，此时作用和wait相同；
        3.pid = 0时，等待同一个进程组中的任何子进程；
        4.pid < -1时，等待一个指定进程组中的任何子进程，这个进程组的ID等于pid
的绝对值。
    options:options提供了一些额外的选项来控制waitpid
        WNOHANG    : 若子进程仍然在运行，则返回0
        （注意只有设置了这个标志，waitpid才有可能返回0）
        WUNTRACED : 如果子进程由于传递信号而停止，则马上返回。
        （只有设置了这个标志，waitpid返回时，其WIFSTOPPED(status)才有可能返
回true）
    &status参数：
        WIFEXITED(status)
        如果正常退出（exit）返回非零值；这时可以用WEXITSTATUS(status) 得到退
出编号（exit的参数）
        WIFSIGNALED(status)
        如果异常退出 （子进程接受到退出信号） 返回非零值；使用WTERMSIG
(status) 得到使子进程退出得信号编号
        WIFSTOPPED(status)
        如果是暂停进程返回的状态，返回非零值；使用WSTOPSIG（status） 得到使子
进程暂停得信号编号
    */

    while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED)) > 0)
// 以非阻塞方式等待所有子进程，若成功回收了子进程，则返回这个子进程的PID，
&status中返回其状态
    {

        // 如果当前这个子进程的job已经删除了，则表示有错误发生
        if ((job = getjobpid(jobs, pid)) == NULL)
        {
            printf("Lost track of (%d)\n", pid);
```

```
            return;
        }

        jid = job->jid;
        // 接下来判断三种状态
        //  如果这个子进程收到了一个暂停信号（还没退出）
        if (WIFSTOPPED(status))
        {
            printf("Job [%d] (%d) stopped by signal %d\n", jid,
job->pid, WSTOPSIG(status));
            // 使用WSTOPSIG（status） 得到使子进程暂停得信号编号
            job->state = ST; // 状态设为挂起
        }
        // 如果子进程通过调用 exit 或者一个返回 (return) 正常终止
        else if (WIFEXITED(status))
        {
            if (deletejob(jobs, pid))
                if (verbose)
                {
                    printf("sigchld_handler: Job [%d] (%d)
deleted\n", jid, pid);
                    printf("sigchld_handler: Job [%d] (%d)
terminates OK (status %d)\n", jid, pid, WEXITSTATUS(status));
                    // 用WEXITSTATUS(status) 得到退出编号（exit的参数）
                }
        }
        // 如果子进程是因为一个未被捕获的信号终止的，例如SIGKILL
        else
        {
            if (deletejob(jobs, pid))
            { // 清除进程
                if (verbose)
                    printf("sigchld_handler: Job [%d] (%d)
deleted\n", jid, pid);
            }
            printf("Job [%d] (%d) terminated by signal %d\n", jid,
pid, WTERMSIG(status));
            // 使用WTERMSIG (status) 得到使子进程退出得信号编号
        }
    }

    if (verbose)
```

```
        puts("sigchld_handler: exiting");

    return;
}

/*
 * sigint_handler - 当用户在键盘上键入ctrl-c时，内核会向shell发送一个
SIGINT。抓住它并将其发送到前台作业。
 */
void sigint_handler(int sig)
{
    if (verbose)
        puts("sigint_handler: entering");
    pid_t pid = fgpid(jobs); // 获取前台进程的pid

    if (pid)
    {
        // 发送SIGINT给前台进程组里的所有进程
        // 需要注意的是，前台进程组内的进程除了当前前台进程以外，还包括前台进程的
子进程。
        // 最多只能存在一个前台进程，但前台进程组内可以存在多个进程
        if (kill(-pid, SIGINT) < 0) // 采用负数发送信号到进程组，使该进程
终止
            unix_error("kill (sigint) error");
        if (verbose)
        {
            printf("sigint_handler: Job (%d) killed\n", pid);
        }
    }
    if (verbose)
        puts("sigint_handler: exiting");
    return;
}

/*
 * sigtstp_handler - 每当用户在键盘上键入ctrl-z时，内核都会向shell发送一个
sigtstp。捕获它并通过向它发送SIGTSTP来挂起前台作业。
 */
void sigtstp_handler(int sig)
{
    if (verbose)
        puts("sigstp_handler: entering");
```

```
    pid_t pid = fgpid(jobs);                    // 获取前台进程的pid号
    struct job_t *job = getjobpid(jobs, pid); // 获取前台进程的job本体

    if (pid)

        if (kill(-pid, SIGTSTP) < 0) // 采用负数发送信号到进程组，使该进程
挂起
            unix_error("kill (tstp) error");
    if (verbose)
    {
        printf("sigstp_handler: Job [%d] (%d) stopped\n", job->jid,
pid);
    }

    if (verbose)
        puts("sigstp_handler: exiting");
    return;
}

/*********************
 * End signal handlers
 *********************/

/***********************************************
 * Helper routines that manipulate the job list
 **********************************************/

/* clearjob - Clear the entries in a job struct */
void clearjob(struct job_t *job)
{
    job->pid = 0;
    job->jid = 0;
    job->state = UNDEF;
    job->cmdline[0] = '\0';
}

/* initjobs - Initialize the job list */
void initjobs(struct job_t *jobs)
{
    int i;
```

```
    for (i = 0; i < MAXJOBS; i++)
        clearjob(&jobs[i]);
}

/* maxjid - Returns largest allocated job ID */
int maxjid(struct job_t *jobs)
{
    int i, max = 0;

    for (i = 0; i < MAXJOBS; i++)
        if (jobs[i].jid > max)
            max = jobs[i].jid;
    return max;
}

/* addjob - Add a job to the job list */
int addjob(struct job_t *jobs, pid_t pid, int state, char *cmdline)
{
    int i;

    if (pid < 1)
        return 0;

    for (i = 0; i < MAXJOBS; i++)
    {
        if (jobs[i].pid == 0)
        {
            jobs[i].pid = pid;
            jobs[i].state = state;
            jobs[i].jid = nextjid++;
            if (nextjid > MAXJOBS)
                nextjid = 1;
            strcpy(jobs[i].cmdline, cmdline);
            if (verbose)
            {
                printf("Added job [%d] %d %s\n", jobs[i].jid,
jobs[i].pid, jobs[i].cmdline);
            }
            return 1;
        }
    }
    printf("Tried to create too many jobs\n");
```

```
    return 0;
}

/* deletejob - Delete a job whose PID=pid from the job list */
int deletejob(struct job_t *jobs, pid_t pid)
{
    int i;

    if (pid < 1)
        return 0;

    for (i = 0; i < MAXJOBS; i++)
    {
        if (jobs[i].pid == pid)
        {
            clearjob(&jobs[i]);
            nextjid = maxjid(jobs) + 1;
            return 1;
        }
    }
    return 0;
}

/* fgpid - Return PID of current foreground job, 0 if no such job
*/
pid_t fgpid(struct job_t *jobs)
{
    int i;

    for (i = 0; i < MAXJOBS; i++)
        if (jobs[i].state == FG)
            return jobs[i].pid;
    return 0;
}

/* getjobpid  - Find a job (by PID) on the job list */
struct job_t *getjobpid(struct job_t *jobs, pid_t pid)
{
    int i;

    if (pid < 1)
        return NULL;
```

```
    for (i = 0; i < MAXJOBS; i++)
        if (jobs[i].pid == pid)
            return &jobs[i];
    return NULL;
}

/* getjobjid  - Find a job (by JID) on the job list */
struct job_t *getjobjid(struct job_t *jobs, int jid)
{
    int i;

    if (jid < 1)
        return NULL;
    for (i = 0; i < MAXJOBS; i++)
        if (jobs[i].jid == jid)
            return &jobs[i];
    return NULL;
}

/* pid2jid - Map process ID to job ID */
int pid2jid(pid_t pid)
{
    int i;

    if (pid < 1)
        return 0;
    for (i = 0; i < MAXJOBS; i++)
        if (jobs[i].pid == pid)
        {
            return jobs[i].jid;
        }
    return 0;
}

/* listjobs - Print the job list */
void listjobs(struct job_t *jobs)
{
    int i;

    for (i = 0; i < MAXJOBS; i++)
    {
        if (jobs[i].pid != 0)
```

```
        {
            printf("[%d] (%d) ", jobs[i].jid, jobs[i].pid);
            switch (jobs[i].state)
            {
            case BG:
                printf("Running ");
                break;
            case FG:
                printf("Foreground ");
                break;
            case ST:
                printf("Stopped ");
                break;
            default:
                printf("listjobs: Internal error: job[%d].state=%d
", i, jobs[i].state);
            }
            printf("%s", jobs[i].cmdline);
        }
    }
}
/******************************
 * end job list helper routines
 ******************************/

/***********************
 * Other helper routines
 ***********************/

/*
 * usage - print a help message
 */
void usage(void)
{
    printf("Usage: shell [-hvp]\n");
    printf("   -h   print this message\n");
    printf("   -v   print additional diagnostic information\n");
    printf("   -p   do not emit a command prompt\n");
    exit(1);
}

/*
```

```
 * unix_error - unix-style error routine
 */
void unix_error(char *msg)
{
    fprintf(stdout, "%s: %s\n", msg, strerror(errno));
    exit(1);
}

/*
 * app_error - application-style error routine
 */
void app_error(char *msg)
{
    fprintf(stdout, "%s\n", msg);
    exit(1);
}

/*
 * Signal - wrapper for the sigaction function
 */
handler_t *Signal(int signum, handler_t *handler)
{
    struct sigaction action, old_action;

    action.sa_handler = handler;
    sigemptyset(&action.sa_mask); /* block sigs of type being
handled */
    action.sa_flags = SA_RESTART; /* restart syscalls if possible
*/

    if (sigaction(signum, &action, &old_action) < 0)
        unix_error("Signal error");
    return (old_action.sa_handler);
}

/*
 * sigquit_handler - The driver program can gracefully terminate
the
 *    child shell by sending it a SIGQUIT signal.
 */
void sigquit_handler(int sig)
{
```

```
    printf("Terminating after receipt of SIGQUIT signal\n");
    exit(1);
}
```

5 学习参考

CSDN：

https://blog.csdn.net/qq_51684393/article/details/124888703

CSAPP：第八章-异常控制流